An OMG® Unified POS Retail Peripheral Architecture Publication

OBJECT MANAGEMENT GROUP

Unified POS Retail Peripheral Architecture

Version 1.15

OMG Formal Document Number: formal/2020-01-05
Standard document URL: https://www.omg.org/spec/UPOS/

Normative Machine Readable Files:

https://www.omg.org/spec/UPOS/20181204/BeltEvent/BeltEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/BillAcceptorEvent/BillEventAcceptorV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/BillAcceptor/BillAcceptorV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/BillDispenser/BillDispenserV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/BiometricsEvent/BiometricsEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/Biometrics/BiometricsV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204BumpBarEvent/BumpBarEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/BumpBar/BumpBarV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CashChangerEvent/CashChangerEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CashChanger/CashChangerV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CashDrawerEvent/CashDrawerEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CashDrawer/CashDrawerV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CATEvent/CATEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CAT/CATV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CheckScannerEvent/CheckScannerEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CheckScanner/CheckScannerV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CoinAcceptorEvent/CoinAcceptorEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CoinAcceptor/CoinAcceptorV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/CoinDispenserEvent/CoinDispenserEventV1.15.0.xsd

https://www.omg.org/spec/UPOS/20181204/CoinDispenser/CoinDispenserV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ElectronicJournalEvent/ElectronicJournalEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ElectronicJournal/ElectronicJournalV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ElectronicValueRWEvent/ElectronicValueRWEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ElectronicValueRW/ElectronicValueRWV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/FiscalPrinterEvent/FiscalPrinterEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/FiscalPrinter/FiscalPrinterV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/GateEvent/GateEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/GateEvent/GateEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204HardTotalsEvent/HardTotalsEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/HardTotals/HardTotalsV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ImageScannerEvent/ImageScannerEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ImageScanner/ImageScannerV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204ItemDispenserEvent/ItemDispenserEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ItemDispenser/ItemDispenserV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/KeylockEvent/KeylockEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/Keylock/KeylockV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/LightsEvent/LightsEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/Lights/LightsV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/LineDisplayEvent/LineDisplayEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/LineDisplay/LineDisplayV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MICREvent/MICREventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MICR/MICRV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MotionSensorEvent/MotionSensorEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MotionSensor/MotionSensorV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MSREvent/MSREventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/MSR/MSRV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/PINPadEvent/PINPadEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/PINPad/PINPadV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/PointCardRWEvent/PointCardRWEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/PointCardRW/PointCardRWV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/POSKeyboardEvent/POSKeyboardEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/POSKeyboard/POSKeyboardV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/POSPowerEvent/POSPowerEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/POSPrinterEvent/POSPrinterEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/POSPrinter/POSPrinterV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/RemoteOrderDisplayEvent/RemoteOrderDisplayEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/RemoteOrderDisplay/RemoteOrderDisplayV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/RFIDScannerEvent/RFIDScannerEventV1.15.0.xsd

https://www.omg.org/spec/UPOS/20181204/RFIDScanner/RFIDScannerV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ScaleEvent/ScaleEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/Scale/ScaleV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/SignatureCaptureEvent/SignatureCaptureEventV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/SignatureCapture/SignatureCaptureV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/SmartCardRWEvent/SmartCardRWV1.15.0.xsd
https://www.omg.org/spec/UPOS/20181204/ToneIndicatorEvent/ToneIndicatorEventV1.15.0.xsd

https://www.omg.org/spec/UPOS/20181204/ToneIndicator/ToneIndicatorV1.15.0.xsd

Copyright © 2011-2019 Object Management Group

Use of Specification - Terms, Conditions & Notices

The material in this document details an Object Management Group specification in accordance with the terms, conditions and notices set forth below. This document does not represent a commitment to implement any portion of this International Standard in any company's products. The information contained in this document is subject to change without notice.

Licenses

The companies listed above have granted to the Object Management Group, Inc. (OMG) a nonexclusive, royalty-free, paid up, worldwide license to copy and distribute this document and to modify this document and distribute copies of the modified version. Each of the copyright holders listed above has agreed that no person shall be deemed to have infringed the copyright in the included material of any such copyright holder by reason of having used the specification set forth herein or having conformed any computer software to the specification.

Subject to all of the terms and conditions below, the owners of the copyright in this International Standard hereby grant you a fully-paid up, non-exclusive, nontransferable, perpetual, worldwide license (without the right to sublicense), to use this International Standard to create and distribute software and special purpose specifications that are based upon this International Standard, and to use, copy, and distribute this International Standard as provided under the Copyright Act; provided that: (1) both the copyright notice identified above and this permission notice appear on any copies of this International Standard; (2) the use of the specifications is for informational purposes and will not be copied or posted on any network computer or broadcast in any media and will not be otherwise resold or transferred for commercial purposes; and (3) no modifications are made to this International Standard. This limited permission automatically terminates without notice if you breach any of these terms or conditions. Upon termination, you will destroy immediately any copies of the specifications in your possession or control.

Patents

The attention of adopters is directed to the possibility that compliance with or adoption of OMG specifications may require use of an invention covered by patent rights. OMG shall not be responsible for identifying patents for which a license may be required by any OMG specification, or for conducting legal inquiries into the legal validity or scope of those patents that are brought to its attention. OMG specifications are prospective and advisory only. Prospective users are responsible for protecting themselves against liability for infringement of patents.

General Use Restrictions

Any unauthorized use of this International Standard may violate copyright laws, trademark laws, and communications regulations and statutes. This document contains information which is protected by copyright. All Rights Reserved. No part of this work covered by copyright herein may be reproduced or used in any form or by any means--graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems--without permission of the copyright owner.

Disclaimer Of Warranty

WHILE THIS PUBLICATION IS BELIEVED TO BE ACCURATE, IT IS PROVIDED "AS IS" AND MAY CONTAIN ERRORS OR MISPRINTS. THE OBJECT MANAGEMENT GROUP AND THE COMPANIES LISTED ABOVE MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS PUBLICATION, INCLUDING BUT NOT LIMITED TO ANY WARRANTY OF TITLE OR OWNERSHIP, IMPLIED WARRANTY OF MERCHANTABILITY OR WARRANTY OF FITNESS FOR A PARTICULAR PURPOSE OR USE.

IN NO EVENT SHALL THE OBJECT MANAGEMENT GROUP OR ANY OF THE COMPANIES LISTED ABOVE BE LIABLE FOR ERRORS CONTAINED HEREIN OR FOR DIRECT, INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, RELIANCE OR COVER DAMAGES, INCLUDING LOSS OF PROFITS, REVENUE, DATA OR USE, INCURRED BY ANY USER OR ANY THIRD PARTY IN CONNECTION WITH THE FURNISHING, PERFORMANCE, OR USE OF THIS MATERIAL, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

The entire risk as to the quality and performance of software developed using this International Standard is borne by you. This disclaimer of warranty constitutes an essential part of the license granted to you to use this International Standard.

Restricted Rights Legend

Use, duplication or disclosure by the U.S. Government is subject to the restrictions set forth in subparagraph (c) (1) (ii) of The Rights in Technical Data and Computer Software Clause at DFARS 252.227-7013 or in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clauses at 48 C.F.R. 52.227-19 or as specified in 48 C.F.R. 227-7202-2 of the DoD F.A.R. Supplement and its successors, or as specified in 48 C.F.R. 12.212 of the Federal Acquisition Regulations and its successors, as applicable. The specification copyright owners are as indicated above and may be contacted through the Object Management Group, 109 Highland Avenue, Needham, MA 02494, U.S.A.

Trademarks

CORBA®, CORBA logos®, FIBO®, Financial Industry Business Ontology®, FINANCIAL INSTRUMENT GLOBAL IDENTIFIER®, IIOP®, IMM®, Model Driven Architecture®, MDA®, Object Management Group®, OMG®, OMG Logo®, SoaML®, SOAML®, SysML®, UAF®, Unified Modeling Language®, UML®, UML Cube Logo®, VSIPL®, and XMI® are registered trademarks of the Object Management Group, Inc.

For a complete list of trademarks, see: http://www.omg.org/legal/tm_list.htm. All other products or company names mentioned are used for identification purposes only, and may be trademarks of their respective owners.

Compliance

The copyright holders listed above acknowledge that the Object Management Group (acting itself or through its designees) is and shall at all times be the sole entity that may authorize developers, suppliers and sellers of computer software to use certification marks, trademarks or other special designations to indicate compliance with these materials.

Software developed under the terms of this license may claim compliance or conformance with this International Standard if and only if the software compliance is of a nature fully matching the applicable compliance points as stated in the specification. Software developed only partially matching the applicable compliance points may claim only that the software was based on this International Standard, but may not claim compliance or conformance with this International Standard. In the event that testing suites are implemented or approved by Object Management Group, Inc., software developed using this International Standard may claim compliance or conformance with the specification only if the software satisfactorily completes the testing suites.

OMG's Issue Reporting Procedure

All OMG specifications are subject to continuous review and improvement. As part of this process we encourage readers to report any ambiguities, inconsistencies, or inaccuracies they may find by completing the Issue Reporting Form listed on the main web page https://www.omg.org, under Documents, Report a Bug/Issue (https://issues.omg.org/issues/create-new-issue).

Table of Contents

Preface xvii
1 Introduction and Architecture 1
1.1 What is Unified POS? 1
1.2 Conformance 5
1.3 Architectural Overview 6
2 Common Properties, Methods, and Events 35
2.1 General 35
2.2 Summary 35
2.3 General Information 37
2.4 Properties (UML attributes) 39
2.5 Methods (UML operations) 48
2.6 Events (UML interfaces) 60
3 Belt 67
3.1 General 67
3.2 Summary 67
3.3 General Information 70
3.4 Belt Class Diagram 70
3.5 Belt Sequence Diagram 71
3.6 Properties (UML attributes) 75
3.7 Methods (UML operations) 82
3.8 Events (UML interfaces) 84
4 Bill Acceptor 87
4.1 General 87
4.2 Summary 87
4.3 General Information 90
4.4 Bill Acceptor Class Diagram 91
4.5 Properties (UML attributes) 95
4.6 Method (UML operations) 99
4.7 Events (UML interfaces) 103
5 Bill Dispenser 107
5.1 General 107
5.2 Summary 107
5.3 General Information 110
5.4 Properties (UML attributes) 114
5.5 Methods (UML operations) 118

5.6 Events (UML interfaces) 121
6 Biometrics 123
6.1 General 123
6.2 Summary 123
6.3 General Information 127
6.4 Properties (UML Attributes) 135
6.5 Methods (UML operations) 143
6.6 Events (UML Interfaces) 149
7 Bump Bar 153
7.1 General 153
7.2 Summary 153
7.3 General Information 156
7.4 Properties (UML attributes) 161
7.5 Methods (UML operations) 165
7.6 Events (UML interfaces) 169
8 Cash Changer 173
8.1 General 173
8.2 Summary 173
8.3 General Information 176
8.4 Properties (UML attributes) 185
8.5 Methods (UML operations) 197
8.6 Events (UML interfaces) 204
9 Cash Drawer 207
9.1 General 207
9.2 Summary 207
9.3 General Information 209
9.4 Properties (UML attributes) 212
9.5 Methods (UML operations) 214
9.6 Events (UML interfaces) 215
10 CAT - Credit Authorization Terminal 217
10.1 General 217
10.2 Summary 217
10.3 General Information 221
10.4 Properties (UML attributes) 231
10.5 Methods (UML operations) 250
10.6 Events (UML interfaces) 260
11 Check Scanner 265
11.1 General 265
11.2 Summary 265
11.3 General Information 268

11.4 Properties (UML attributes) 274
11.5 Methods (UML operations) 288
11.6 Events (UML interfaces) 296
12 Coin Acceptor 301
12.1 General 301
12.2 Summary 301
12.3 General Information 304
12.4 Properties (UML attributes) 309
12.5 Methods (UML operations) 314
12.6 Events (UML interfaces) 317
13 Coin Dispenser 321
13.1 General 321
13.2 Summary 321
13.3 General Information 323
13.4 Properties (UML attributes) 327
13.5 Methods (UML operations) 328
13.6 Events (UML interfaces) 330
14 Electronic Journal 333
14.1 General 333
14.2 Summary 333
14.3 General Information 336
14.4 Properties (UML Attributes) 342
14.5 Methods (UML operations) 348
14.6 Events (UML interfaces) 357
15 Electronic Value Reader/Writer 361
15.1 General 361
15.2 Summary 361
15.3 General Information 367
15.4 Properties (UML attributes) 393
15.5 Methods (UML operations) 427
15.6 Events (UML interfaces) 464
16 Fiscal Printer 473
16.1 General 473
16.2 Summary 473
16.3 General Information 482
16.4 Properties (UML attributes) 500
16.5 Methods (UML operations) 537
16.6 Events 607

17 Gate 613
17.1 General 613
17.2 Summary 613
17.3 General Information 615
17.4 Properties (UML attributes) 618
17.5 Methods (UML operations) 619
17.6 Events (UML interfaces) 620
18 Hard Totals 623
18.1 General 623
18.2 Summary 623
18.3 General Information 626
18.4 Properties (UML attributes) 631
18.5 Methods (UML operations) 633
18.6 Events (UML interfaces) 643
19 Image Scanner 645
19.1 General 645
19.2 Summary 645
19.3 General Information 648
19.4 Properties (UML attributes) 657
19.5 Methods (UML operations) 664
19.6 Events (UML interfaces) 665
20 Item Dispenser 669
20.1 General 669
20.2 Summary 669
20.3 General Information 672
20.4 Properties (UML attributes) 675
20.5 Methods (UML operations) 677
20.6 Events (UML interfaces) 678
21 Keylock 681
21.1 General 681
21.2 Summary 681
21.3 General Information 683
21.4 Properties (UML attributes) 686
21.5 Methods (UML operations) 689
21.6 Events (UML interfaces) 690
22 Lights 693
22.1 General 693
22.2 Summary 693
22.3 General Information 695
22.4 Properties (UML attributes) 699

22.5 Methods (UML operations) 700
22.6 Events (UML interfaces) 702
23 Line Display 703
23.1 General 703
23.2 Summary 703
23.3 General Information 707
23.4 Properties (UML attributes) 712
23.5 Methods (UML operations) 735
23.6 Events (UML interfaces) 750
24 MICR - Magnetic Ink Character Recognition Reader 753
24.1 General 753
24.2 Summary 753
24.3 General Information 756
24.4 Properties (UML attributes) 761
24.5 Methods (UML operations) 765
24.6 Events (UML interfaces) 769
25 Motion Sensor 773
25.1 General 773
25.2 Summary 773
25.3 General Information 775
25.4 Properties (UML attributes) 779
25.5 Methods (UML operations) 779
25.6 Events (UML interfaces) 780
26 MSR - Magnetic Stripe Reader 783
26.1 General 783
26.2 Summary 783
26.3 General Information 787
26.4 Properties (UML attributes) 798
26.5 Methods (UML operations) 823
26.6 Events (UML interfaces) 828
27 Pin Pad 833
27.1 General 833
27.2 Summary 833
27.3 General Information 836
27.4 Properties (UML attributes) 843
27.5 Methods (UML operations) 854
27.6 Events (UML interfaces) 858
28 Point Card Reader/Writer 861
28.1 General 861
28.2 Summary 861

28.3 General Information 866
28.4 Properties (UML attributes) 877
28.5 Methods (UML operations) 897
28.6 Events (UML Interfaces) 904
29 POS Keyboard 909
29.1 General 909
29.2 Summary 909
29.3 General Information 911
29.4 Properties (UML attributes) 915
29.5 Events (UML interfaces) 916
30 POS Power 921
30.1 General 921
30.2 Summary 921
30.3 General Information 924
30.4 Properties (UML attributes) 938
30.5 Methods (UML operations) 945
30.6 Events (UML interfaces) 948
31 POS Printer 951
31.1 General 951
31.2 Summary 951
31.3 General Information 958
31.4 Properties (UML attributes) 982
31.5 Methods (UML operations) 1032
31.6 Events (UML interfaces) 1071
32 Remote Order Display 1079
32.1 General 1079
32.2 Summary 1079
32.3 General Information 1083
32.4 Properties (UML attributes) 1089
32.5 Methods (UML operations) 1099
32.6 Events (UML interfaces) 1116
33 RFID Scanner 1121
33.1 General 1121
33.2 Summary 1121
33.3 General Information 1124
33.4 Properties (UML Attributes) 1132
33.5 Methods (UML operations) 1136
33.6 Events (UML Interfaces) 1142
34 Scale 1147
34.1 General 1147

34.2 Summary 1147
34.3 General Information 1150
34.4 Scale Class Diagram 1152
34.5 Scale Sequence Diagram 1153
34.6 Properties (UML attributes) 1155
34.7 Methods (UML operations) 1164
34.8 Events (UML interfaces) 1176

35 Scanner (Bar Code Reader) 1179

35.1 General 1179
35.2 Summary 1179
35.3 General Information 1181
35.4 Properties (UML attributes) 1184
35.5 Events (UML interfaces) 1190

36 Signature Capture 1193

36.1 General 1193
36.2 Summary 1193
36.3 General Information 1196
36.4 Properties (UML attributes) 1200
36.5 Methods (UML operations) 1203
36.6 Events (UML interfaces) 1204

37 Smart Card Reader/Writer 1209

37.1 General 1209
37.2 Summary 1209
37.3 General Information 1212
37.4 Smart Card Reader / Writer Class Diagram 1213
37.5 Model 1213
37.6 Card Insertion Diagram 1215
37.7 Device Sharing 1216
37.8 Data Transfer Modes 1216
37.9 Smart Card Reader / Writer Sequence Diagram 1217
37.10 Smart Card Reader / Writer State Diagram 1218
37.11 Properties (UML Attributes) 1218
37.12 Methods (UML operations) 1224
37.13 Events (UML Interfaces) 1229

38 Tone Indicator 1233

38.1 General 1233
38.2 Summary 1233
38.3 General Information 1236
38.4 Properties (UML attributes) 1239
38.5 Methods (UML operations) 1244
38.6 Events (UML interfaces) 1245

Annexes

Annex A - OLE for Retail POS - OPOS Implementation Reference. 1249
A.1 What is OLE for Retail POS? 1249
A.2 Who Should Read This Section 1250
A.3 General OLE for Retail POS Control Model 1250
A.4 OPOS Definitions 1251
A.5 How an Application Uses an OPOS Control 1251
A.6 When Methods and Properties May Be Accessed 1252
A.7 Status, Result Code, and State Model 1253
A.8 Device Sharing Model 1256
A.9 Events 1257
A.10 OPOS Event Registration Sequence Diagram 1258
A.11 Input Model 1259
A.12 Output Model 1261
A.13 Device Power Reporting Model 1261
A.14 Device Information Reporting Model 1263
A.15 Update Firmware Device Model 1264
A.16 OPOS Component Descriptions 1265
A.17 Section 1: OPOS Data Types 1266
A.18 Section 2: OPOS Interface Descriptions 1267
A.19 OPOS Common Properties, Methods, and Events 1268
A.20 Common Properties 1268
A.21 Common Methods 1269
A.22 OPOS Programmatic Names 1269
A.23 Properties 1271
A.24 Methods 1286
A.25 Events 1297
A.26 Peripheral Interfaces 1300
A.27 OPOS: Cash Drawer 1301
A.28 OPOS: MICR 1305
A.29 Section 3: OPOS Registry Usage 1310
A.30 Section 4: OPOS Application Header Files 1315
A.31 Section 5: Technical Details 1316
A.32 Section 6: Release 1.5 API Change: ClaimDevice and ReleaseDevice 1318
A.33 Section 7: OPOS APG Change History 1319
A.34 Section 8: OPOS Control Programmer's Guide 1329

Annex B - Java for Retail POS - JavaPOS Implementation Reference 1361
B.1 What is Java for Retail POS? 1361
B.2 Benefits 1361
B.3 Dependencies 1362
B.4 Relationship to OPOS 1362
B.5 Who Should Read This Section 1362
B.6 Appendix Overview 1363
B.7 Architectural Overview 1363
B.8 Architectural Components 1363
B.9 Device Behavior Models 1365
B.10 Introduction to Properties, Methods, and Events 1365
B.11 Device Initialization and Finalization 1366
B.12 Device Sharing Model 1367
B.13 Data Types 1368
B.14 Exceptions 1369
B.15 Events 1371
B.16 JavaPOS Event Registration Sequence Diagram 1374
B.17 Device Input Model 1376
B.18 Device Output Models 1378
B.19 Device Power Reporting Model 1379
B.20 Device Information Reporting Model 1382
B.21 Update Firmware Device Model 1383
B.22 Device States 1383
B.23 Threads 1384
B.24 Version Handling 1384
B.25 Classes and Interfaces 1384
B.26 Synopsis 1384
B.27 Sample Class and Interface Hierarchies 1387
B.28 Sample Application Code 1389
B.29 Package Structure 1390
B.30 Device Controls 1399
B.31 Device Control Responsibilities 1399
B.32 Device Service Management 1400
B.33 Property and Method Forwarding 1402
B.34 Event Handling 1403
B.35 Device Control Version Handling 1404
B.36 Device Services 1406
B.37 Device Service Responsibilities 1406
B.38 Property and Method Processing 1407
B.39 Event Generation 1407
B.40 Physical Device Access 1408
B.41 API Mapping Rules 1408
B.42 JavaPOS Component Descriptions 1408

B.43 Section 1: JavaPOS Data Types 1408
B.44 Section 2: JavaPOS Interface Descriptions 1410
B.45 JavaPOS Common Properties, Methods, and Events 1410
B.46 Properties 1413
B.47 Methods 1422
B.48 Events 1432
B.49 Peripheral Interfaces 1436
B.50 Section 3: Technical Details - OPOS and JavaPOS 1442
B.51 OPOS to JavaPOS - API Mapping Rules 1442
B.52 API Deviations 1443
B.53 Mapping of CharacterSet 1444
B.54 Handling Binary Data inside Strings 1445
B.55 Section 4: JavaPOS Change History 1446

Annex C - POS for .NET Implementation Reference 1453
C.1 What is POS for .NET? 1453
C.2 Who Should Read This Section 1454
C.3 Overview of POS for .NET 1454
C.4 POS for .NET Definitions 1455
C.5 Key POS for .NET Features 1456
C.6 Key Programming Construct Differences from OPOS 1458
C.7 Key Parameter Differences 1481
C.8 Key Property Signature Differences 1483
C.9 PosExplorer API 1483
C.10 Service Object Registry 1486
C.11 Consuming Service Objects 1486
C.12 Writing Service Objects 1487
C.13 Status, State Model, and Exceptions 1488
C.14 Device Sharing Model 1489
C.15 Events 1490
C.16 Input Model 1491
C.17 Output Model 1492
C.18 Device Power Reporting Model 1493
C.19 Power Reporting Properties 1494
C.20 Device Information Reporting Model 1495
C.21 POS for .NET Component Descriptions 1496
C.22 Common Properties 1499
C.23 Common Methods 1506
C.24 Common Events 1519
C.25 POS for .NET vs. UnifiedPOS Members 1520
C.26 Interim Procedure Available For Legacy OPOS Services...
Shim Code Usage 1521
C.27 Architecture Structures 1521

C.28 Method of Implementation 1523
C.29 Method of Administration 1525

Annex D - XMLPOS - XML POS Mapping Reference 1529
D.1 Overview 1529
D.2 Referenced Documents 1530
D.3 Taxonomy for Conversion from UnifiedPOS to XML 1530
D.4 Changes to XMLPOS 1530
D.5 XMLPOS Architecture Overview 1531
D.6 UnifiedPOS XML Errors 1536
D.7 XMLPOS Common Events 1541
D.8 XMLPOS Common Properties 1543
D.9 XMLPOS Common Data 1544
D.10 ARTS Common Data 1545
D.11 UnifiedPOS Devices 1545
D.12 NAFEM Protocol 1741
D.13 Distributed Files 1751
D.14 Glossary 1752

Annex E - Change History 1753
E.1 Release Version 1.4 1753
E.2 Release Version 1.5 1753
E.3 Release Version 1.6 1754
E.4 Release Version 1.7 1755
E.5 Release Version 1.8 1760
E.6 Release Version 1.9 1762
E.7 Release Version 1.10 1764
E.8 Release Version 1.11 1766
E.9 Release Version 1.12 1769
E.10 Release Version 1.13 1771
E.11 Release Version 1.14 1775
E.12 Release Version 1.14.1 1778
E.13 Release Version 1.15 1780

Annex F - Additional Software References 1781
F.1 General 1781
F.2 UML References 1781

Annex G - Additional Hardware References 1783
G.1 General 1783
G.2 USB PlusPower Connector 1783

Annex H - Deprecation History 1787
H.1 General 1787

Annex I - Systems Management Information 1789
I.1 What is Systems Management? 1789
I.2 How is UnifiedPOS involved in Systems Management? 1789
I.3 Who Should Read This Section 1789
I.4 UnifiedPOS Device Information Reporting Model 1790
I.5 Architectural Overview 1792
I.6 Utilized CIM Data Types 1795
I.7 Common Properties, Methods, and Events 1796
I.8 Common Methods 1797
I.9 Properties 1797
I.10 Peripheral Interfaces 1797
I.11 Technical Details 1903

Annex J - Device Statistics 1905
J.1 General 1905

Preface

About the Object Management Group

OMG

Founded in 1989, the Object Management Group, Inc. (OMG) is an open membership, not-for-profit computer industry standards consortium that produces and maintains computer industry specifications for interoperable, portable and reusable enterprise applications in distributed, heterogeneous environments. Membership includes Information Technology vendors, end users, government agencies and academia.

OMG member companies write, adopt, and maintain its specifications following a mature, open process. OMG's specifications implement the Model Driven Architecture® (MDA®), maximizing ROI through a full-lifecycle approach to enterprise integration that covers multiple operating systems, programming languages, middleware and networking infrastructures, and software development environments. OMG's specifications include: UML® (Unified Modeling Language™); CORBA® (Common Object Request Broker Architecture); CWM™ (Common Warehouse Metamodel); and industry-specific standards for dozens of vertical markets.

More information on the OMG is available at http://www.omg.org/.

OMG Specifications

As noted, OMG specifications address middleware, modeling and vertical domain frameworks. All OMG Specifications are available from this URL:

http://www.omg.org/spec

All of OMG's formal specifications may be downloaded without charge from our website. (Products implementing OMG specifications are available from individual suppliers.) Copies of specifications, available in PDF format, may be obtained from the Specifications Catalog cited above or by contacting the Object Management Group, Inc. at:

OMG Headquarters
109 Highland Avenue
Needham, MA 02494
USA
Tel: +1-781-444-0404
Fax: +1-781-444-0320
Email: pubs@omg.org

Certain OMG specifications are also available as ISO standards. Please consult http://www.iso.org

Issues

The reader is encouraged to report any technical or editing issues/problems with this specification to (https://issues.omg.org/issues/create-new-issue).

UnifiedPOS Technical Committee Members

Bizerba GmbH & Co. KG

Datalogic Scanning, Inc.

Epson America, Inc.

Fujitsu Frontech Limited

IBM Corporation

Microsoft Corporation

NCR Corporation

OPOS-Japan

Seiko Epson Corporation

Sorimachi Giken CO, LTD

Star Micronics, CO. LTD

Toshiba Global Commerce Solutions, Inc.

Wincor Nixdorf International GmbH.

1 Introduction and Architecture

1.1 What is Unified POS?

UnifiedPOS is the acronym for **Unified** **P**oint **o**f **S**ervice. It is an architectural specification for application interfaces to point-of-service devices that are used in the retail environment. This standard is both operating system independent and language neutral and defines:

- An architecture for application interface to retail devices.
- A set of retail device behaviors sufficient to support a range of POS solutions.

The UnifiedPOS standard will include:

- The UnifiedPOS Retail Peripheral Architecture overview.
- Text descriptions of the interface to the functions of the device.
- UML terminology and diagrams for each device category, to describe:
  - Relationships between classes/interfaces and objects in the system.
- Basis for creating C++, Java, IDL, or other OO technology to implement the UML design.
- Operational characteristics and details for implementations which are compliant to the UnifiedPOS architecture. These were added in the Appendices for UnifiedPOS starting in Version 1.6. As new Implementations become available, additional Appendices will be added in future versions of the standard.

The UnifiedPOS standard will **not** include:

- Specific language API specifications.
- Complete software components. Hardware providers, software providers, or third-party providers develop and distribute these components.
- Certification mechanism; this must be handled by individual language standard committees (such as the OLE for Retail POS (OPOS), POS for .NET, and Java for Retail POS (JavaPOS) committees).

1.1.1 About This Documentation

Updated in Release 1.12

Since the release of UnifiedPOS Version 1.4, the Retail Standards' committees had been maintaining three separate standard documents, OPOS, JavaPOS and UnifiedPOS. The architecture and device characteristics are identical in each of these documents. The addition of new device categories and/or enhancements to existing chapters required consultation and agreement on the technical content for the each of the separate standards. However, in addition to that technical work, there is a heavy administrative burden in generating the correct documentation for three different versions of the standard's specification. That process was inherently error prone in that the same changes had to be maintained in multiple documents. Confusion has resulted in cases where differences have inadvertently appeared in the documentation.

In order to simplify the process and bring a higher quality of review to ongoing modifications of the documentation, the UnifiedPOS standard committee made a change in the process for documenting its requirements. Beginning with UnifiedPOS Version 1.6, only the UnifiedPOS document was updated and the structure of the documentation was

changed. The main body of the documentation includes the abstracted generic description of all device categories plus additional general design and utilization guidelines. Specific reference platform requirements are now found in the included annexes that outline the implementation information for each of the specific existing implementations, such as OPOS, JavaPOS and POS For Dot Net. *(Note: OPOS-J, the POS Standards body from Japan, has and plans to continue to maintain a translated Japanese version of the OPOS documentation for their developer community.)*

The documentation is arranged in such a fashion that allows the new user to gather a general education about the UnifiedPOS Standard by reading the "Introduction and Architecture" section. This section is designed to give an overview of the material covered in the entire standard and provide an outline of the design features that must be adhered to for a developer to implement the standard. For a first time reader, this section should be read and understood, as it will make the remaining chapters and appendices more beneficial. For a familiar user, this section may serve as a "fall-back" reference for clarification of the requirements when developing a Device Service or usage of the Device Services by an Application.

Chapter 2 outlines the Properties, Methods, and Events that are Common to all peripheral devices. It is important to understand this section and make reference to it when questions arise on the common functionality that apply to all device classes.

The following chapters define each of the POS peripheral devices that are covered in the standard. The specific Properties, Methods, and Events that are peculiar to the peripheral are defined. Any additional helpful information relevant to the POS peripheral are also included. As new POS peripherals are added a new chapter will be added to describe the devices unique requirements.

Following the chapters describing the POS peripheral devices, annexes are included that outline specific details on implementation dependencies for each of the supported Operating Systems and/or language specific development platforms.

"Annex A" includes the definition, goals, and deliverables for OPOS. There are explanations for the input/output and device sharing for Microsoft's COM model for the operation of the interface. Event and error handling unique to this implementation is described. It concludes with a version change history that guides the user in understanding the evolution of the OPOS implementation of the standard.

"Annex B" includes the definition, goals, and deliverables for JavaPOS. There are explanations for the input/output and device sharing for the Java model for the operation of the interface. Event and error handling unique to this implementation is included. It also concludes with a version change history that is helpful to the user to understand the evolution of the JavaPOS implementation requirements.

"Annex C" includes the definition, goals, and deliverables for POS for .NET. There are explanations for the input/output and device sharing for Microsoft's .NET model for the operation of the interface and the differences from the OPOS COM architecture that affect implementation. Event and error handling unique to this implementation are described. It also includes a version change history section and brief clarifications of the design philosophy.

"Annex D" is included to provide information on the usage of XML for peripheral message mapping. Future versions of the UnifiedPOS standard will evolve to a greater dependence upon XML as the command and interoperability infrastructure of choice. There is increasing interest and focus on using XML for communicating with peripheral devices. It opens up many new possibilities for creating Device Services that, when coupled with Universal Plug and Play hardware connection technologies such as USB, will provide for true language and operating system independence.

"Annex E" incorporates an overall Change History for the documentation. It is highly recommended that the experienced user refer to this section as an aide for understanding the version to version documentation changes as a resource to help in the updating of the device support and/or implementation changes necessary to the software for efficient usage.

"Annex F" provides some additional software reference material that may prove helpful to the understanding of the principals and documentation constructs that the UnifiedPOS standard incorporates. The developer is encouraged to check this section as additional resource material will be added as the standard evolves from version to version.

"Annex G" includes additional hardware reference material that is pertinent to the hardware design for compliance to the UnifiedPOS standard. The USB Plus Power connector recommendations are outlined in this section as well.

"Annex H" provides information on functionality and changes that are documented in the UnifiedPOS standard in a version that will cause a previously defined function to be deprecated. While every attempt is made to minimize the use of Deprecation, the reader is highly encouraged to review this section to ensure a firm understanding of direction the standard is evolving.

"Annex I" includes the definition, goals, and deliverables for Systems Management. Appendix I is targeted at a systems management solution developer who requires access to POS-specific device information. It is also targeted to the system developer who will provide device information from within the Services he provided.

"Annex J" includes the definitions and deliverables for UnifiedPOS Device Statistics. This information was previously issued in a separate document, but starting with v1.12, the device statistics appendix was added as an appendix to the specification

1.1.2 Goals

The goals of UnifiedPOS are to provide:

- Common device architecture that is international and extends across vendors, platforms, and retail format.
- Standards for application to device interfaces in an operating system independent and language neutral manner.
- Reduced implementation costs for vendors to support multiple (for example, Windows/COM, Windows/.NET, and Java) platforms because they share the same architecture. This should produce speed to market for innovation.
- An environment avoiding competition between standards while encouraging competition among implementations.

1.1.3 Dependencies

Success of the goals of UnifiedPOS depends upon platform specific standard committees (such as JavaPOS and OLE for Retail POS (OPOS) technical committees) to advance the architecture into platform specific documentation, API definitions and implementations.

The specific technical implementations require:

- Platform specific implementation references. (See Annexes A, B, C, & D.)
- Source files, including:
  - Definition files. Various interface and class files described in the standard.
  - Example files. These will include a set of sample Control classes, to illustrate the interface presented to an application.

1.1.4 UnifiedPOS Relationship to Conforming Platform Mappings

The UnifiedPOS specification formalizes and documents the underlying retail device architecture, shared by the JavaPOS, OPOS, and POS for .NET standards, in an operating system independent and language neutral manner. The first release of the UnifiedPOS Specification was Version 1.4.

The JavaPOS, OPOS, and POS for .NET standards have been established as conformant platform mappings of the UnifiedPOS specification. In UnifiedPOS Version 1.6, appendices were added in order to document specific implementation details for each of these platforms. JavaPOS will be recognized as the only UnifiedPOS conformant, operating system neutral, Java language mapping (See Annex B). OPOS will be recognized as the only UnifiedPOS conformant language neutral COM mapping (See Annex A). POS for .NET will be recognized as the only UnifiedPOS conformant language neutral .NET mapping (See Annex C). Future UnifiedPOS mappings to platforms other than Java, COM, and .NET will be included as appendices to the UnifiedPOS specification as they become available.

This acceptance of the existing standards is based on their close conformance to a common design model. Historically, the OPOS standards provided device interfaces for Win32-based terminals using ActiveX technologies. The OPOS standard was used as the starting point for JavaPOS, due to:

- **Similar purposes.** Both standards involved developing device interfaces for a segment of the software community.
- **Reuse of device models.** The majority of the OPOS documentation specifies the properties, methods, events, and constants used to model device behavior. These behaviors are in large part independent of programming language.
- **Reduced learning curve.** Many application and hardware vendors are already familiar with using and implementing the OPOS APIs.

Therefore, retail application developers and Service writers can continue to write their code in conformance with one or both of the JavaPOS or OPOS standards. The content of the UnifiedPOS specification, however, along with the appropriate annex, will constitute the definition of how an application can be developed to meet the UnifiedPOS standard. The standards committees do not intend to release future versions of the specific OPOS and JavaPOS documents after the Version 1.6 specification.

The UnifiedPOS specification is also the basis for the POS for .NET implementation, which similarly adheres to this common approach for the access and control of POS peripherals.

1.1.5 Who Should Read This Document

The UnifiedPOS Architecture is targeted to the standard committees that will provide the language specific mapping and Programmer's Guides. However, the application developer who will use POS devices, the system developer who will write POS device code, and the suppliers of POS devices for retail may be interested in the device characteristics as portrayed in this document.

This guide assumes that the standard committee member is familiar with the following:

- General characteristics of POS peripheral devices.
- UnifiedPOS terminology and architecture.
- UML for reading the design.

1.2 Conformance

1.2.1 Unified POS

The UnifiedPOS specification formalizes and documents the underlying retail device architecture, shared by JavaPOS, OPOS, and POS for .NET, which provide standard platform specific mappings of the UnifiedPOS specification. JavaPOS, OPOS and POS for .NET also provide base classes and/or interfaces to be used for implementations of UnifiedPOS conformant device interfaces. To be UnifiedPOS conformant POS applications and device vendors have to provide implementation using an appropriate platform-specific mapping.

1.2.1.1 ARTS IP Policy

This specification was originally created under the ARTS IP Policy which can be found here: https://www.omg.org/cgi-bin/doc?retail/2017-12-01

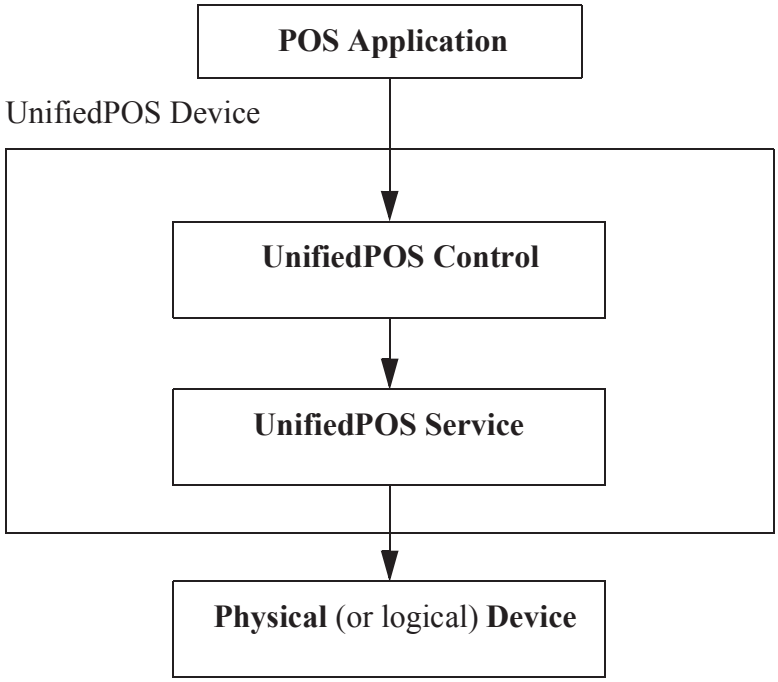
Summary Points

1. The Policy is applicable to all members of ARTS and acceptance of this Policy will be a condition of ARTS membership. Non-members wishing to attend technical meetings must agree in writing to accept the Policy.
2. The Policy is applicable to the Data Model, ARTS XML, UnifiedPOS and future technical committees established by the ARTS Board to develop specifications.
3. The Policy permits members that disclose intellectual property to reserve rights on how they will license its use.
4. The Policy encourages members to immediately disclose upon discovery of intellectual property that maybe embedded in ARTS specifications.
5. No member is required to conduct patent searches to search for intellectual property within ARTS specification(s.)
6. Members who participate in the development of ARTS specifications must assign representatives with reasonable knowledge in the field of work.
7. The Policy establishes defined periods for review of developing draft specifications for both technical accuracy and intellectual property. A public review period is also provided.
8. Members who do not disclose intellectual property within an ARTS specification before that specification is approved by the ARTS Board, must provide a 12-month royalty-free license to all implementers, during which time ARTS may modify the specification to remove the infringing IP and each implementer may make appropriate resolution.
9. There is a default reasonable and non-discriminatory (“RAND”) licensing obligation for members of Work teams and Technical Committees with only limited exceptions.UnifiedPOS specification formalizes and documents the underlying retail device.

1.3 Architectural Overview

1.3.1 General

UnifiedPOS defines a multi-layered architecture in which a POS Application interacts with the Physical or Logical Device through the UnifiedPOS Control layer.



1.3.2 Architectural Components

The **POS Application** (or **Application**) is an Application that uses one or more UnifiedPOS devices.

UnifiedPOS Devices are divided into categories called **Device Categories**, such as Cash Drawer and POS Printer.

Each UnifiedPOS Device is a combination of these components:

- **Control** for a device category. The Control class provides the interface between the Application and the device category. It contains no graphical component and is therefore invisible at runtime.

  The Control has been designed so that all implementations of a device category's control will be compatible. Therefore, the Control can be developed independently of the Service for the same device category (they can even be developed by different companies).

- **Service**, which is a component called by the Control through the **Service Interface**. The Service is used by the Control to implement UnifiedPOS-prescribed functionality for a Physical Device. It can also call special event methods provided by the Control to deliver events to the Application.

  A set of Service classes can be implemented to support Physical Devices with multiple Device Categories.

The Application manipulates the **Physical Device** (the hardware unit or peripheral) by calling the platform specific APIs which conform to the UnifiedPOS standard. Some Physical Devices support more than one device category. For example, some POS Printers include a Cash Drawer kickout, and some Bar Code Scanners include an integrated Scale. However with UnifiedPOS, an application treats each of these device categories as if it were an independent Physical Device. The UnifiedPOS Device standard developer is responsible for presenting the peripheral in this way.

Note: Occasionally, a Device may be implemented in software with no user-exposed hardware, in which case it is called a **Logical Device**.

1.3.3 Use of UML

The UnifiedPOS standard includes the use of UML terminology and diagrams to define device categories. Following is a brief description of the extensions to UML to make it better fit the UnifiedPOS architecture (this extension is expected and allowed by the UML, see Booch98 reference in the "UML References" in Annex D).

Should any discrepancies exist between the UML diagrams and the specification text, then the text takes precedence.

Name	Applies to UML Symbol	Meaning
<<capability>>	**Class attribute**	**stereotype** which flags the attribute as a UnifiedPOS capability
<<prop>>	**Class attribute**	**stereotype** which flags the attribute as a UnifiedPOS property
<<event>>	**Class**	**stereotype** to indicate that the class/interface will be mapped to a UnifiedPOS event which in turn is mapped to a JavaPOS event class or a COM event for OPOS or a .NET event
exclusive-use	**Class**	**constraint** that indicates this Device Service or Service Object follows the exclusive-use behavior defined in the UnifiedPOS documentation in section "Exclusive-Use Devices" in Chapter 1.
sharable	**Class**	**constraint** that indicates this Device Service or Service Object follows the sharable behavior defined in the UnifiedPOS documentation in section "Sharable Devices" in Chapter 1.
read-only **read-write**	**Class attribute**	**constraint** that indicates the mutability of the attribute. For example, in JavaPOS, read-only attributes translate to having a getter method for the attribute and read-write attributes have getter and setter methods for attributes.

Name	Applies to UML Symbol	Meaning
access after <open>| <open-claim>| <open-enable>| <open-claim-enable>	**Class attribute**	**constraint** that indicates this attribute is accessible when the service is in the state indicated. For example {access after opened-claim-enable} indicates that the attribute is accessible when the service has been opened, claimed and enabled in the order indicated.
raises-exception	**Class operation**	**constraint** that indicates this method can throw an exception if the implementation language supports exception; otherwise, some mechanism is used to notify the application that an invalid condition occurred. A value is returned to indicate the error.
use after <open>| <open-claim>| <open-enable>| <open-claim-enable>	**Class operation**	**constraint** that indicates this operation is accessible when the service is in the state indicated. For example {use after open-claim-enable} indicates that the method is accessible when the service has been opened, claimed and enabled in the order indicated.

1.3.3.1 Package Diagram

UnifiedPOS uses Static Structure Diagrams to define common interfaces.

Note: This package diagram is included to give some logical structure to the interfaces in the UnifiedPOS interfaces UML diagrams. Some implementations may have a corresponding equivalence for the packages and some may not. Also, note that the name ‘upos’ may be replaced by an implementation specific prefix (e.g., JavaPOS uses Java packages and maps the prefix ‘upos’ to ‘jpos’).

1.3.4 Data Types

Updated in Release 1.13

UnifiedPOS uses textual references to data types which will be defined for specific language usage:

UnifiedPOS	JavaPOS	OPOS	POS for .NET	UML	UnifiedPOS text Usage
boolean	boolean	BOOL	bool	in *boolean*	Boolean true or false.
boolean by reference	boolean[1]	BOOL*	Not used **	inout *boolean*	Mutable boolean.
binary	byte[]	BSTR	byte[]	in *binary*	Immutable array of bytes.
binary by reference	byte[1][]	BSTR*	Not used **	inout *binary*	Mutable array of bytes. (Both its size and contents may be modified.)
array of binary	byte[][]	SAFEARRAY of BSTR	Not used **	in *binary[]*	Immutable array of array of bytes.
byte	byte	LONG	byte	in *byte*	8-bit integer. (See HardTotals, **setAll** method.)
int32	int	LONG	int or enum	in *int32*	32-bit integer.
int32 array	int[]	SAFEARRAY of LONG	int[]	in *int32 array*	Immutable array of 32-bit integers.
int32 array by reference	int[1][]	SAFEARRAY* of LONG	Not used **	inout *int32* array	Mutable array of 32-bit integers. (Both its size and contents may be modified.)
int32 by reference	int[1]	LONG*	Not used **	inout *int32*	Mutable 32-bit integer.
currency	long	CURRENCY or CY	decimal	in *currency*	64-bit integer. Sometimes used for currency values where 4 decimal places are implied. E.g., if the integer is “1234567”, then the currency value is “123.4567”. See footnote[a]
currency by reference	long[1]	CURRENCY* or CY*	Not used **	inout *currency*	Mutable 64-bit integer.
string	String	BSTR	string	in *string*	Text character string. See footnote[b]
string by reference	String[1]	BSTR*	Not used **	inout *string*	Mutable text character string. (Both its size and contents may be modified.)
array of points	Point[]	BSTR	Point[]	inout *point[]*	Immutable array of points. Used by Signature Capture.
object	Object	BSTR*	object	inout *object*	An object. This will usually be subclassed to provide a Service-specific parameter.
nls	String	LONG	CultureInfo	in *nls*	Operating System National Language Support data type.

a. Six decimal place precision is required for all computations in conversion between currencies but is not required for the representation of the solution.
b. For data elements within comma delimited string data, no leading or trailing whitespace is permitted, unless that whitespace is part of the data element. Comma delimited string data is typically used for a series of numbers, in which no whitespace should be included in the string.

For Java:
The convention of *type*[1] (an array of size 1) is used to pass a mutable basic type. This is required since Java's primitive types, such as **int** and **boolean**, are passed by value, and its primitive wrapper types, such as **Integer** and **Boolean**, do not support modification. For strings and arrays, do not use a null value to report no information. Instead use an empty string ("") or an empty array (zero length). In some chapters, an integer may contain a "bit-wise mask." That is, the integer data may be interpreted one or more bits at a time. The individual bits are numbered beginning with Bit 0 as the least significant bit.
** **POS for .NET** does not use "out" parameters, return values are used instead.

1.3.5 Device Behavior Models

1.3.5.1 Introduction to Properties, Methods, and Events

An application accesses a POS Device via platform specific APIs.

The three elements of UnifiedPOS standard for APIs are:

- **Properties.** Properties are device characteristics or settings. A type is associated with each property, such as ***boolean*** or ***string***. An application may retrieve a property's value, and it may set a writable property's value.
- **Methods.** An application calls a method to perform or initiate some activity at a device. Some methods require parameters of specified types for sending and/or returning additional information.
- **Events.** A Device implementation may call back into the application via events. The application may need to register for events. The mechanism to do this is implementation specific.

Properties (UML Attributes)

Note: For each interface a UML listing of the properties and methods of the interface will be included in a table. The properties are indicated as attributes. The generic UML naming pattern for attributes is the following:

visibility Name: type-expression = default-value { property-string }

where:

visibility in this document is always public for application visible interfaces but is not explicitly shown.

Name is the name of the attribute

type-expression is the type of the attribute, which is one of UnifiedPOS types defined in Intro-8.

default-value[1] the default value of the attributes in UML, (optional)

1. Not used by UnifiedPOS standard

property-string property value to apply to the element. For attributes, we define two such strings: read-only and read-write, which indicates the mutability of the attribute.

An example of a property attribute is as follows:

DeviceEnabled: ***boolean*** **{ read-write }**

Methods (UML Operations)

The generic UML pattern for methods is the following:

visibility name (parameter-list): return-type-expr { property string }

where:

parameter - list is a comma separated list of formal parameters using the following generic UML naming pattern:

kind name: type-expression (= default-value)[1]

where:

kind is either: 'in,' 'out,' or 'inout' with the default set to 'in' if absent

property-string is a property string to apply to the element. For methods an additional property string called 'raises-exception' is defined which means that this method can throw the exception if the implementation language supports exception; otherwise, some mechanism is used to notify the application that an invalid condition occurred.

An example of a method operation is as follows:

open (logicalDeviceName: ***string*** **): void { raises-exception }**

Events (UML Interfaces)

Events are being modeled as UML classes which will possibly contain attributes stereotyped with the event stereotype. The generic UML pattern for events is a UML box with the stereotype <<event>> (class diagram) with the event name and a list of the properties. This representation is different from Properties and Methods.

<<event>>
xxEvent

where:

XxxEvent stands for the UnifiedPOS event name and the second compartment of the box would contain a list of attributes for the event.

---

1. ***default-value*** is not used by the UnifiedPOS standard

1.3.5.2 Device Initialization and Finalization

Updated in Release 1.11

Initialization

The first actions that an application must take to use a Device are:

- Obtain a reference to a Control,
- Prepare Control for the events that the application needs to receive, if necessary.

To initiate activity with the Physical Device, an application calls the Control's **open** method:

The *logicalDeviceName* parameter specifies a logical device to associate with the Device. The **open** method performs the following steps:

- Creates and initializes an instance of the proper Service class for the specified name.
- Initializes many of the properties, including the descriptions and version numbers of the Device.

More than one instance of a Control may have a Physical Device open at the same time. Therefore, after the Device is opened, an application might need to call the **claim** method to gain exclusive access to it. Claiming the Device ensures that other Control instances do not interfere with the use of the Device. An application can **release** the Device to share it with another Control instance– for example, at the end of a transaction.

Before using the Device, an application must set the **DeviceEnabled** property to true. This value brings the Physical Device to an operational state, while false disables it. For example, if a Scanner Device is disabled, the Physical Device will be put into its non-operational state (when possible). Whether physically operational or not, any input is discarded until the Device is enabled.

Initialization and Error Reporting

Added in Release 1.11

Error conditions may require that a Service fail during one or more of the initialization APIs - **open**, **claim**, and/or **DeviceEnabled**=true. The following are recommendations for initialization-time error handling by Service implementers. These guidelines are not mandated, however, because of the wide variation in some hardware devices and their initialization requirements, and due to variations in already released Services.

open *Primary purpose*: Initialize the software stack, including the creation of the Service and initialization of its supporting software components.

1. The Service must fail an **open** API call if software initialization fails.
   Example: Supporting software components are not installed or available, so fail the API call.
2. If the Service must probe the device in order to correctly set open-time properties (such as capabilities), then the Service should fail an **open** API call if it cannot access the device.
   Example: A Service supports several line display models and sets the UnifiedPOS capabilities after communicating with the device. If the device's port is not available or the device does not respond, then the Service cannot complete its **open** work and will need to fail the API call.
3. For other cases, the Service should succeed the **open** API call and report a failure (if needed) later.

Example: A Service cannot open an RS232 port during **open**. If the previous case (#2) above does not apply, then the Service should succeed the **open** and report the port open failure during **claim**, if the port is still not available.

claim *Primary purpose*: Acquire exclusive access to the device, for exclusive-use devices.

1. The Service must fail a **claim** API call if another process has claimed the device and the claim timeout expires.
2. If the device is not accessible, then the Service should fail a **claim** API call.
   Examples: A required communications or I/O port cannot be opened or claimed. The Service determines that the device is not present or is offline. For each of these cases, the Service should fail the API call.

For other cases, the Service should succeed the **claim** API call. This specifically includes cases where runtime faults exist.
Examples: A POSPrinter receipt station is out-of-paper, or the POSPrinter receipt station detects a printer jam. These are runtime faults that occur from time to time during operation, and are user correctable. The Service should succeed the **claim**. POSPrinter runtime faults should be reported (after **DeviceEnabled**=true) by **StatusUpdateEvent**s and/**or** by exceptions from APIs such as **printNormal**.

DeviceEnabled = **true** *Primary purpose*: Final preparation for operation and application use.

1. If the device is not accessible, then the Service should fail a **DeviceEnabled**= true API call. (Note that the device may have been accessible at **claim** but is now inaccessible.)
   Example: The Service determines that the device is not present or is offline, so the Service should fail the API call.
2. For other cases, the Service should succeed the **DeviceEnabled**=true API call. This specifically includes cases where runtime faults exist.
   Examples: See **claim** case (#3) above.

An application developer must be prepared for failures at any of the initialization points. With the variations in hardware devices and in their Service implementations, a well-written application will respond predictably to the widest range of error conditions and their reporting as possible.

Retail devices may communicate with a POS terminal using a wide variety of ports, including RS232, RS485, Parallel, USB, Ethernet, and Wireless. In addition, devices may be powered directly by the terminal or by an external power source. These guidelines may be applied to all of these devices. Two examples with typical initialization follow.

Example 1: Hand-held scanner attached to a terminal's powered RS232 port.

- **open**: Succeed if software initialization is successful.
- **claim**: Succeed if **open** was successful and if an attempt to communicate with the device is successful.
- **DeviceEnabled = true**: Succeed if **claim** was successful and if an attempt to communicate with the device is successful.
- **While enabled**: If the device is unplugged from the powered RS232 port, then detect the power state change and report to the application. If the device is later plugged back in, then detect the power state change and report to the application. For many devices, power state changes can be accomplished by monitoring the RS232 DSR signal. (Note that hot unplugging and plugging in with this port type is probably not recommended by the hardware vendor.)

Example 2: Deck scanner/scale attached to a terminal's USB port, powered by a “brick.”

- **open**: Succeed if software initialization is successful.
- **claim**: Succeed if **open** was successful and if an attempt to communicate with the device is successful.

- **DeviceEnabled = true**: Succeed if claim was successful and if an attempt to communicate with the device is successful.
- **While enabled**: If the device is unplugged from the USB port or from its power source, then detect the power state change and report to the application. If the device is later plugged back in, then detect the power state change and report to the application. An operating system-specific mechanism detects power state changes, such as an open, write, or read failure with specific failure statuses.

Notice that the general initialization handling is very similar, even though the second example will typically require somewhat more logic within the Service to monitor and re-initialize the device connection.

Finalization

After an application finishes using the Physical Device, it should call the **close** method. If the **DeviceEnabled** property is true, **close** disables the Device. If the **Claimed** property is true, **close** releases the claim on the device.

Before exiting, an application should close all open Devices to free device resources in a timely manner.

Summary

In general, an application follows this general sequence to open, use, and close a Device:

Obtain a Control reference.

Prepare for events if necessary.

Call the **open** method to instantiate a Service and link it to the Control.

Call the **claim** method to gain exclusive access to the Physical Device. Required for exclusive-use Devices; optional for some sharable Devices. (See “Device Sharing Model” on page 14 for more information).

Set the **DeviceEnabled** property to true to make the Physical Device operational. (For sharable Devices, the Device may be enabled without first **claim**ing it.)

Use the device.

Set the **DeviceEnabled** property to false to disable the Physical Device.

Call the **release** method to release exclusive access to the Physical Device.

Call the **close** method to unlink the Service from the Control.

Release events receipt if necessary

Remove the reference to the Control

1.3.6 Device Sharing Model

Devices fall into two sharing categories:

- Devices that are to be used exclusively by one Control instance.
- Devices that may be partially or fully shared by multiple Control instances.

Any Physical Device may be open by more than one Control instance at a time. However, activities that an application can perform with a Control may be restricted to the Control instance that has claimed access to the Physical Device.

1.3.6.1 Exclusive-Use Devices

The most common device type is called an **exclusive-use device**. An example is the POS printer. Due to physical or operational characteristics, an exclusive-use device can only be used by one Control at a time. An application must call the Device's **claim** method to gain exclusive access to the Physical Device before most methods, properties, or events are legal. Until the Device is claimed and enabled, calling methods or accessing properties may cause a failure condition to occur.

An application may in effect share an exclusive-use device by calling the Control's **claim** method before a sequence of operations, and then calling the **release** method when the device is no longer needed. While the Physical Device is released, another Control instance can claim it.

When an application calls the **claim** method again (assuming it did not perform the sequence of **close** method followed by **open** method on the device), some settable device characteristics are restored to their condition at the **release**. Examples of restored characteristics are the line display's brightness, the MSR's tracks to read, and the printer's characters per line. However, state characteristics are not restored, such as the printer's sensor properties. Instead, these are updated to their current values.

1.3.6.2 Sharable Devices

Some devices are **sharable devices**. An example is the keylock. A sharable device allows multiple Control instances to call its methods and access its properties. Also, it may deliver its events to multiple Controls. A sharable device may still limit access to some methods or properties to the Control that has claimed it, or it may deliver some events only to the Control that has claimed it.

1.3.6.3 Events

Updated in Release 1.12

UnifiedPOS architecture uses events to inform the application of various activities or changes with the Device. The five event types follow.

Event Class	Description	Supported When A Device Category Supports...
DataEvent	Input data has been placed into device class-category properties.	Event-driven input
ErrorEvent	An error has occurred during event-driven input or asynchronous output.	Event-driven input -or- Asynchronous output
OutputCompleteEvent	An asynchronous output has successfully completed.	Asynchronous output

StatusUpdateEvent	A change in the Physical Device's status has occurred. Devices may be able to report device power state. See "Device Power Reporting Model," page 22.	Status change notification
DirectIOEvent	This event may be defined by a Service provider for purposes not covered by the specification.	Always, for Service-specific use

The Service must enqueue these events on an internally created and managed queue. All events are delivered in a first-in, first-out manner. (The only exception is that a special input error event is delivered early if some data events are also enqueued. See "Device Input Model," page 18.) Events are delivered by an internally created and managed Service thread. The Service causes event delivery by calling an event firing callback method in the Control, which then delivers the event to the application.

The following conditions cause event delivery to be delayed until the condition is corrected:

- The application has set the property **FreezeEvents** to true.
- The event type is a **DataEvent** or an input **ErrorEvent**, but the property **DataEventEnabled** is false. (See "Device Input Model," page 18.)

Unless specified otherwise, properties that convey device state information (e.g., **JrnEmpty** and **DrawerOpened**) are kept current while the device is enabled, regardless of the setting of the **FreezeEvents** property.

Rules for event queue management are:

- The Device may only enqueue new events while the Device is enabled.
- The Device delivers enqueued events until the application calls the **release** method (for exclusive-use devices) or the **close** method (for any device), at which time any remaining events are deleted.
- For input devices, the **clearInput** method clears data and input error events.
- For output devices, the **clearOutput** method clears data and output error events.

1.3.6.4 Errors

UnifiedPOS architecture deals with two kinds of errors as discussed in "Methods (UML Operations)" on page 11 and explanation of exceptions:

- Errors that are "invalid or bad invocations" which are recognized by the Service validation of the request. Method invocations and property accesses may be valid or invalid. If the action is invalid, an invalid condition is set and the application is notified in a fashion appropriate to the platform. For specific implementations, OPOS would produce a **ResultCode** other than OPOS_SUCCESS and JavaPOS would produce an exception.
- Errors that are caused by errant device behavior and produce error events.

1.3.6.5 Error Codes

Updated in Release 1.11

This section lists the general meanings of the error code property when an invalid condition occurs. In general, the property and method descriptions in later chapters list error codes only when specific details or information are added to these general meanings. In UML each error code is:

E_xxx : ***int32*** **{ frozen }**

The error code is set to one of the following values:

Value	Meaning
E_CLOSED	An attempt was made to access a closed Device.
E_CLAIMED	An attempt was made to access a Physical Device that is claimed by another Control instance. The other Control must release the Physical Device before this access may be made. For exclusive-use devices, the application will also need to claim the Physical Device before the access is legal.
E_NOTCLAIMED	An attempt was made to access an exclusive-use device that must be claimed before the method or property set action can be used. If the Physical Device is already claimed by another Control instance, then the status E_CLAIMED is returned instead.
E_NOSERVICE	The Control cannot communicate with the Service, normally because of a setup or configuration error.
E_DISABLED	Cannot perform this operation while the Device is disabled.
E_ILLEGAL	An attempt was made to perform an illegal or unsupported operation with the Device, or an invalid parameter value was used.
E_NOHARDWARE	The Physical Device is not connected to the system or is not powered on.
E_OFFLINE	The Physical Device is off-line.
E_NOEXIST	The file name (or other specified value) does not exist.
E_EXISTS	The file name (or other specified value) already exists.
E_FAILURE	The Device cannot perform the requested procedure, even though the Physical Device is connected to the system, powered on, and on-line.
E_TIMEOUT	The Service timed out waiting for a response from the Physical Device, or the Control timed out waiting for a response from the Service.
E_BUSY	The current Service state does not allow this request. For example, if asynchronous output is in progress, certain methods may not be allowed.
E_EXTENDED	A device category-specific error condition occurred. The error condition code is held in an extended error code.
E_DEPRECATED	The requested operation can not be performed since it has been deprecated.

When more than one error code is valid, the most descriptive code should be selected. For example, the closed, claimed, not claimed, and disabled errors must follow this order of error reporting precedence, from higher to lower:

E_CLOSED	The device must be opened.
E_CLAIMED	The device is opened but not claimed. Another application has the device claimed, so it cannot be claimed at this time.
E_NOTCLAIMED	The device is opened but not claimed. No other application has the device claimed, so it can and must be claimed.
E_DISABLED	The device is opened and claimed (if this is an exclusive-use device), but not enabled.

Extended Error Code

The extended error code is set as follows:

- When the error code is E_EXTENDED, the extended error code is set to a device category-specific value, and must match one of the values given in this document under the appropriate device category chapter.
- When the error code is any other value, the extended error code **may** be set by the Service to any Service-specific value. These values are only meaningful if an application adds Service-specific code to handle them.

1.3.6.6 Device Input Model

Updated in Release 1.13

The standard UnifiedPOS input model for exclusive-use devices is event-driven input. Event-driven input allows input data to be received after **DeviceEnabled** is set to true. Received data is enqueued as a **DataEvent**, which is delivered to an application.

If the **AutoDisable** property is true when data is received, then the Device will automatically disable itself, setting **DeviceEnabled** to false. This will inhibit the Device from enqueuing further input and, when possible, physically disable the device.

When the application is ready to receive input from the Device, it sets the **DataEventEnabled** property to true. Then, when input is received (usually as a result of a hardware interrupt), the Device delivers a **DataEvent**. (If input has already been enqueued, the **DataEvent** will be delivered immediately after **DataEventEnabled** is set to true.) The **DataEvent** may include input status information through its Status property. The Device places the input data plus other information as needed into device category-specific properties just before the event is delivered.

Just before delivering this event, the Device disables further data events by setting the **DataEventEnabled** property to false. This causes subsequent input data to be enqueued by the Device while an application processes the current input and associated properties. When an application has finished the current input and is ready for more data, it enables data events by setting **DataEventEnabled** to true.

(Added in 1.13) If an application causes disabling of the device (by setting **DeviceEnabled=false,** or by setting **AutoDisable=true** and a subsequent input event is enqueued), then it may need logic to ignore additional data until it reenables the device. In particular, input that is already received and enqueued will continue to be delivered (unless the **clearInput**, **release** or **close** API is called, at which time undelivered input is discarded). As stated in the **Events** section, the application may control the input delivery by using **the DataEventEnabled** or **FreezeEvents** properties.

Error Handling

Updated in Release 1.12

If the Device encounters an error while gathering or processing event-driven input, then the Device:

- Changes its **State** to S_ERROR.
- Enqueues an **ErrorEvent** with locus EL_INPUT to alert an application of the error condition. This event is added to the end of the queue
- If one or more **DataEvent**s are already enqueued for delivery, an additional **ErrorEvent** with locus EL_INPUT_DATA is enqueued before the **DataEvent**s, as a pre-alert.

This event (or events) is not delivered until the **DataEventEnabled** property is true, so that orderly application sequencing occurs.

Unlike a **DataEvent**, the Device does not disable further **DataEvent**s or input **ErrorEvent**s; it leaves the **DataEventEnabled** property value at true. Note that the application may set **DataEventEnabled** to false within its event handler if subsequent input events need to be disabled for a period of time.

ErrorLocus	Description
EL_INPUT_DATA	Only delivered if the error occurred when one or more **DataEvent**s are already enqueued. This event gives the application the ability to immediately clear the input, or to optionally alert the user to the error before processing the buffered input. This error event is enqueued before the oldest **DataEvent**, so that an application is alerted of the error condition quickly. This locus was created especially for the Scanner: When this error event is received from a Scanner Device, the operator can be immediately alerted to the error so that no further items are scanned until the error is resolved. Then, the application can process any backlog of previously scanned items before error recovery is performed.
EL_INPUT	Delivered when an error has occurred and there is no data available. If some input data was buffered when the error occurred, then an **ErrorEvent** with the locus EL_INPUT_DATA was delivered first, and then this error event is delivered after all **DataEvent**s have been delivered. If the Service has partial data that can be delivered with an **ErrorEvent**, the related data properties should be filled in prior to delivery of the event with this *ErrorLocus*. If there is no partial data to be delivered with the **ErrorEvent**, the data properties should be cleared prior to delivery of this event. **Note**: This EL_INPUT event is not delivered if: an EL_INPUT_DATA event was delivered and the application event handler responded with an ER_CLEAR error response.

The application can cause the *ErrorResponse* property to be set one of the following:

ErrorResponse	Description
ER_CLEAR	Clear the buffered **DataEvent**s and **ErrorEvent**s and exit the error state, changing **State** to S_IDLE. This is the default response for locus EL_INPUT.
ER_CONTINUEINPUT	This response acknowledges the error and directs the Device to continue processing. The Device remains in the error state, and will deliver additional data events as directed by the **DataEventEnabled** property. When all input has been delivered and the **DataEventEnabled** property is again set to true, another **ErrorEvent** is delivered with locus EL_INPUT. This is the default response when the locus is EL_INPUT_DATA, and is legal only with this locus.
ER_RETRY	This response directs the Device to retry the input. The error state is exited, and **State** is changed to S_IDLE. This response may only be selected when the device chapter specifically allows it and when the locus is EL_INPUT. An example is the scale.

The Device exits the Error state when one of the following occurs:

- The application returns from the EL_INPUT **ErrorEvent**.
- The application calls the **clearInput** method.
- The application returns from the EL_INPUT_DATA **ErrorEvent** with *ErrorResponse* set to ER_CLEAR.

Miscellaneous

Updated in Release 1.10

For some Devices, the Application must call a method to begin event driven input. After the input is received by the Device, then typically no additional input will be received until the method is called again to reinitiate input. Examples are the MICR and Signature Capture devices. This variation of event driven input is sometimes called “asynchronous input.”

The **DataCount** property contains the number of **DataEvent**s enqueued by the Device.

Calling the **clearInput** method deletes all input enqueued by a Device. **clearInput** may be called after **open** for sharable devices and after **claim** for exclusive-use devices.

Calling the **clearInputProperties** method sets all data properties, that were populated as a result of firing a **DataEvent** or **ErrorEvent**, back to their default values. This call does not reset the **DataCount** or **State** properties.

The general event-driven input model does not specifically rule out the definition of device categories containing methods or properties that return input data directly. Some device categories define such methods and properties in order to operate in a more intuitive or flexible manner. An example is the Keylock device. This type of input is sometimes called “synchronous input.”

1.3.6.7 Device Output Models

The UnifiedPOS output model consists of two output types: synchronous and asynchronous. A device category may support one or both types, or neither type.

Synchronous Output

The application calls a category-specific method to perform output. The Device does not return until the output is completed; this means the physical device has performed the intended operation. For example the printer has successfully transferred all the output data as ink on the paper.

This type of output is preferred when device output can be performed relatively quickly. Its merit is simplicity.

Asynchronous Output

Updated in Release 1.13

The application calls a category-specific method to start the output. The Device validates the method parameters and produces an error condition immediately if necessary. If the validation is successful, the Device does the following:

1. Buffers the request in program memory, for delivery to the Physical Device as soon as the Physical Device can receive and process it.
2. Sets the **OutputID** property to a unique integer identifier for this request. (For more information about the **OutputID** property, see Chapter 2.)
3. Returns as soon as possible.

When the Device successfully completes a request, an **OutputCompleteEvent** is enqueued for delivery to the application. A property of this event contains the output ID of the completed request. The application should compare the returned **OutputCompleteEvent** property **OutputID** value with the **OutputID** value set by the asynchronous process method call used to send the data in order to track what data has been successfully sent to the device. If the request is terminated before completion, due to reasons such as the application calling the **clearOutput** method or responding to an **ErrorEvent** with a ER_CLEAR response, then no **OutputCompleteEvent** is delivered.

If an error occurs while processing a request, an **ErrorEvent** is enqueued which will be delivered to the application after the events already enqueued, including **OutputCompleteEvent**s (according to the normal Event delivery rules on page 15). No further asynchronous output will occur until the event has been delivered to the application. If the response is ER_CLEAR, then outstanding asynchronous output is cleared. If the response is ER_RETRY, then output is retried; note that if several outputs were simultaneously in progress at the time that the error was detected, then the Service may need to retry all of these outputs.

This type of output is preferred when device output requires slow hardware interactions. Its merit is perceived responsiveness, since the application can perform other work while the device is performing the output.

Note: Asynchronous output is always performed on a first-in first-out basis.

1.3.6.8 Device Power Reporting Model

Updated in Release 1.8

Applications frequently need to know the power state of the devices they use. **Note**: This model is not intended to report Workstation or POS Terminal power conditions (such as “on battery” and “battery low”). Reporting of these conditions is now managed by the POSPower device category.

Model

UnifiedPOS architecture segments device power into three states:

- **ONLINE**. The device is powered on and ready for use. This is the “operational” state.
- **OFF**. The device is powered off or detached from the terminal. This is a “non-operational” state.
- **OFFLINE**. The device is powered on but is either not ready or not able to respond to requests. It may need to be placed online by pressing a button, or it may not be responding to terminal requests. This is a “non-operational” state.

In addition, one combination state is defined:

- **OFF_OFFLINE**. The device is either off or offline, and the Service cannot distinguish these states.

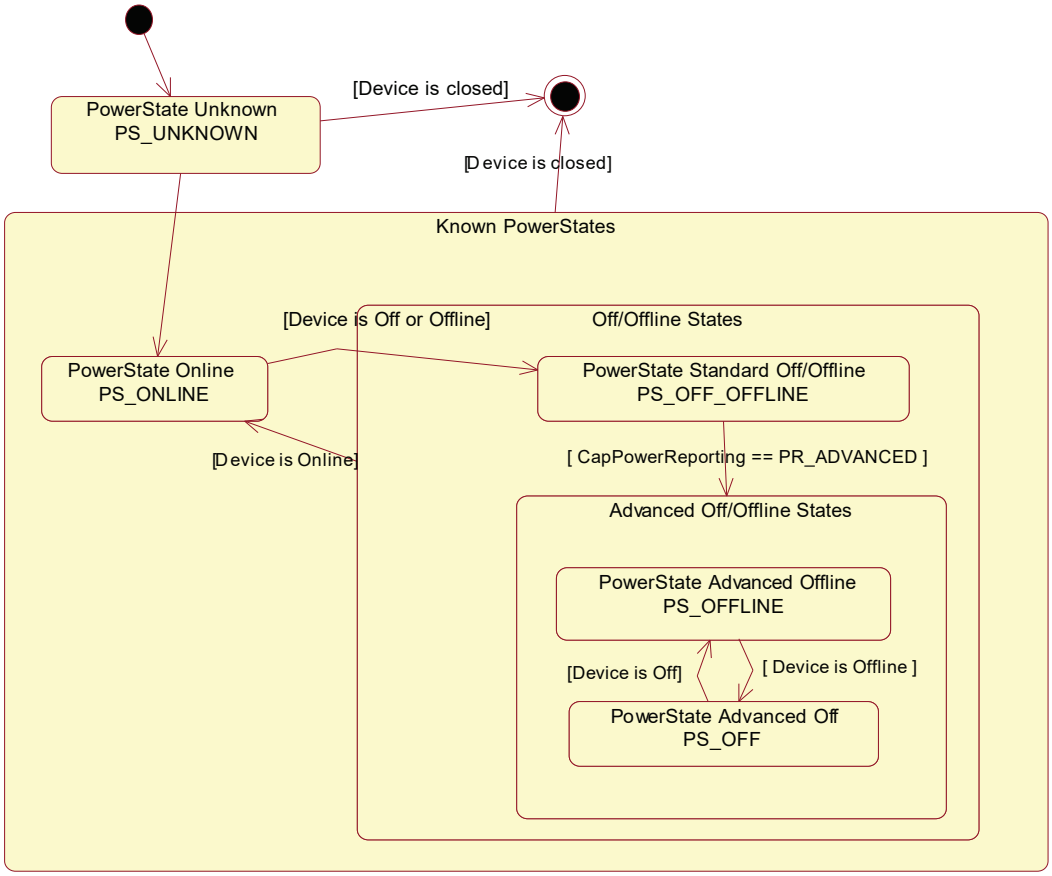
Power reporting only occurs while the device is open, claimed (if the device is exclusive-use), and enabled.

Note - Enabled/Disabled vs. Power States

These states are different and usually independent. UnifiedPOS defines “disabled” / “enabled” as a logical state, whereas the power state is a physical state. A device may be logically “enabled” but physically “offline.” It may also be logically “disabled” but physically “online.” Regardless of the physical power state, UnifiedPOS only reports the state while the device is enabled. (This restriction is necessary because a Service typically can only communicate with the device while enabled.)

If a device is “offline,” then a Service may choose to fail an attempt to “enable” the device. However, once enabled, the Service may not disable a device based on its power state.

1.3.6.9 Power State Diagram



1.3.6.10 Power Properties

The UnifiedPOS device power reporting model adds the following common elements across all device classes.

- **CapPowerReporting** property. Identifies the reporting capabilities of the device. The UML pattern for the property is:

  PR_xxx : *int32* { frozen }

  This property may be one of:

  - PR_NONE. The Service cannot determine the state of the device. Therefore, no power reporting is possible.
  - PR_STANDARD. The Service can determine and report two of the power states - OFF_OFFLINE (that is, off or offline) and ONLINE.
  - PR_ADVANCED. The Service can determine and report all three power states - ONLINE, OFFLINE, and OFF.

- **PowerState** property. Maintained by the Service at the current power condition, if it can be determined. The UML pattern for the property is:

PS_xxx : *int32* { frozen }

This property may be one of:

- PS_UNKNOWN
- PS_ONLINE
- PS_OFF
- PS_OFFLINE
- PS_OFF_OFFLINE

- **PowerNotify** property. The application may set this property to enable power reporting via **StatusUpdateEvent**s and the **PowerState** property. This property may only be changed while the device is disabled (that is, before **DeviceEnabled** is set to true). This restriction allows simpler implementation of power notification with no adverse effects on the application. The application is either prepared to receive notifications or doesn't want them, and has no need to switch between these cases. The UML pattern for the property is:

  PN_xxx : *int32* { frozen }

  This property may be one of:

  - PN_DISABLED
  - PN_ENABLED

1.3.6.11 Power Reporting Requirements for DeviceEnabled

The following semantics are added to **DeviceEnabled** when

CapPowerReporting is not PR_NONE, and
PowerNotify is PN_ENABLED:

- When the Control changes from **DeviceEnabled** false to true, then begin monitoring the power state:
  - If the Physical Device is ONLINE, then **PowerState** is set to PS_ONLINE. A **StatusUpdateEvent** is enqueued with its *Status* property set to SUE_POWER_ONLINE.
  - If the Physical Device's power state is OFF, OFFLINE, or OFF_OFFLINE, then the Service may choose to fail the enable by notifying the application with error code E_NOHARDWARE or E_OFFLINE.

    However, if there are no other conditions that cause the enable to fail, and the Service chooses to return success for the enable, then **PowerState** is set to PS_OFF, PS_OFFLINE, or PS_OFF_OFFLINE.

    A **StatusUpdateEvent** is enqueued with its *Status* property set to SUE_POWER_OFF, SUE_POWER_OFFLINE, or SUE_POWER_OFF_OFFLINE.
  - When the Device changes from **DeviceEnabled** true to false, UnifiedPOS assumes that the Device is no longer monitoring the power state and sets the value of **PowerState** to PS_UNKNOWN.

1.3.6.12 Device Information Reporting Model

Added in Release 1.8

POS Applications, as well as System Management agents, frequently need to monitor the current configuration and usage metrics of the various POS devices that are attached to the POS terminal.

Examples of configuration data are the device's *Serial Number*, *Firmware Version*, and *Connection Type*. Examples of usage data for the POSPrinter device are the *Number of Lines Printed*, *Number of Hours Running*, *Number of paper cuts*, etc. Examples of usage data for the Scanner device are the *Number of scans*, *Number of Hours Running*, etc. Examples of usage data for the MSR device are the *Number of successful swipes*, *Number of swipes resulting in errors*, *Number of Hours Running*, etc. See below for examples of XML definitions of the device statistics accumulated per POS device category.

In some cases, the data may be accumulated and stored within the device itself. In other cases, the data may be accumulated by the Service and stored, possibly on the POS terminal or store controller.

In order for multiple applications (for example a POS application and a System Management application) to obtain statistics from the same device, proper care must be taken by both applications so that the device can be made accessible when required. This is done by using the **claim** method and by setting **DeviceEnabled** to true when access to a device is required and then setting **DeviceEnabled** to false and using the **release** method when access to the device is no longer needed. Coordination of device access via this mechanism is the responsibility of the applications themselves.

Statistics Reporting Properties and Methods

The UnifiedPOS device information reporting model adds the following common properties and methods across all device classes.

- **CapStatisticsReporting** property. Identifies the reporting capabilities of the device. When **CapStatisticsReporting** is false, then no statistical data regarding the device is available. This is equivalent to Services compatible with prior versions of the specification. When **CapStatisticsReporting** is true, then some statistical data for the device is available.
- **CapUpdateStatistics** property. Defines whether gathered statistics (or some of them) can be reset/updated by the application. This property is only valid if **CapStatisticsReporting** is true. When **CapUpdateStatistics** is false, then none of the statistical data can be reset/updated by the application. Otherwise, when **CapUpdateStatistics** is true, then (some of) the statistical data can be reset/updated by the application.
- **resetStatistics** method. Can only be called if both **CapStatisticsReporting** and **CapUpdateStatistics** are true. This method resets one, some, or all of the resettable device statistics to zero.
- **retrieveStatistics** method. Can only be called if **CapStatisticsReporting** is true. This method retrieves one, some, or all of the accumulated statistics for the device.
- **updateStatistics** method. Can only be called if both **CapStatisticsReporting** and **CapUpdateStatistics** are true. This method updates one, some, or all of the resettable device statistics to the supplied values.

XML Definitions for POS Device Statistics

The XML files containing the UnifiedPOS defined statistics for each device category are provided as downloads from the web sites that also host this specification. These statistics can be referenced individually by name or as a group using the "U_" string as (part of) the parameter to the statistics methods.

Manufacturers/Service providers can add their specific statistics in the provided “ManufacturerSpecific” section. These statistics can be referenced individually by name or as a group using the “M_” string as (part of) the parameter to the statistics methods.

The following table contains the definitions of the information contained in the UnifiedPOS defined DeviceInformation section covering all device categories.

<DeviceInformation> XML Definition Name	**Definition description**
UnifiedPOSVersion	Version of the UnifiedPOS specification supported
DeviceCategory	Device category (e.g., POSPrinter)
ManufacturerName	Device manufacturer’s name
ModelName	Device model name
SerialNumber	Device serial number
ManufactureDate	Device manufacture date
MechanicalRevision	Device hardware revision
FirmwareRevision	Device firmware revision
Interface	Device hardware interface (e.g., serial, USB)
InstallationDate	Device installation date

The following is an example of the XML file that describes the “UnifiedPOS” defined statistics for the CashDrawer device category.

```
<?xml version='1.0' ?>
<UPOSStat version="1.13.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.omg.org/UnifiedPOS/namespace/"
xsi:schemaLocation="http://www.omg.org/UnifiedPOS/namespace/UPOSStat.xsd">
   <Event>
      <Parameter>
         <Name>DrawerGoodOpenCount</Name>
         <Value>1353</Value>
      </Parameter>
      <Parameter>
         <Name>DrawerFailedOpenCount</Name>
         <Value>2</Value>
      </Parameter>
      <ManufacturerSpecific>
         <Name>MyPersonalStat</Name>
         <Value>14.32</Value>
         <unitofmeasure>meters</unitofmeasure>
      </ManufacturerSpecific>
   </Event>
```

```
    <Equipment>
        <UnifiedPOSVersion>1.13</UnifiedPOSVersion>
        <DeviceCategory UPOS="CashDrawer"/>
        <ManufacturerName>Cashdrawers R Us</ManufacturerName>
        <ModelName>CD-123</ModelName>
        <SerialNumber>12345</SerialNumber>
        <ManufactureDate>1999-12-31</ManufactureDate>
        <MechanicalRevision>1A</MechanicalRevision>
        <FirmwareRevision>1.0 Rev. B</FirmwareRevision>
        <Interface>RS232</Interface>
        <InstallationDate>2000-03-01</InstallationDate>
    </Equipment>
</UPOSStat>
```

The most up-to-date files defining the XML tag names that conform to the ARTS Data Dictionary and example schemas for the statistics for all device categories can be downloaded from the ARTS web site at http://retail.omg.org

1.3.6.13 Update Firmware Device Model

Added in Release 1.9

POS Applications frequently require the ability to update the firmware in the various POS devices that are attached to the POS terminal. This model defines a consistent application interface for updating the firmware in a device controlled by a UnifiedPOS control.

This model has the following capabilities:

- A property, **CapUpdateFirmware**, that indicates whether a device supports firmware updating.
- A property, **CapCompareFirmwareVersion**, that indicates whether a firmware file's version can be compared against the firmware version of the device.
- A method, **updateFirmware**, to perform an asynchronous update of the firmware in a device.
- A method, **compareFirmwareVersion**, to compare the firmware file's version against the firmware version of the device.
- Additional **StatusUpdateEvent** *Status* values to report the progress of an asynchronous update firmware process.

The update firmware process is an asynchronous operation that reports its progress via **StatusUpdateEvent**s. This update firmware process applies to all device categories defined in UnifiedPOS.

The means by which a Service actually updates the firmware in the device is not covered by this document, only the means by which the update firmware process is started and progress is reported.

1.3.6.14 Device States

UnifiedPOS defines a property **State** with the following values:

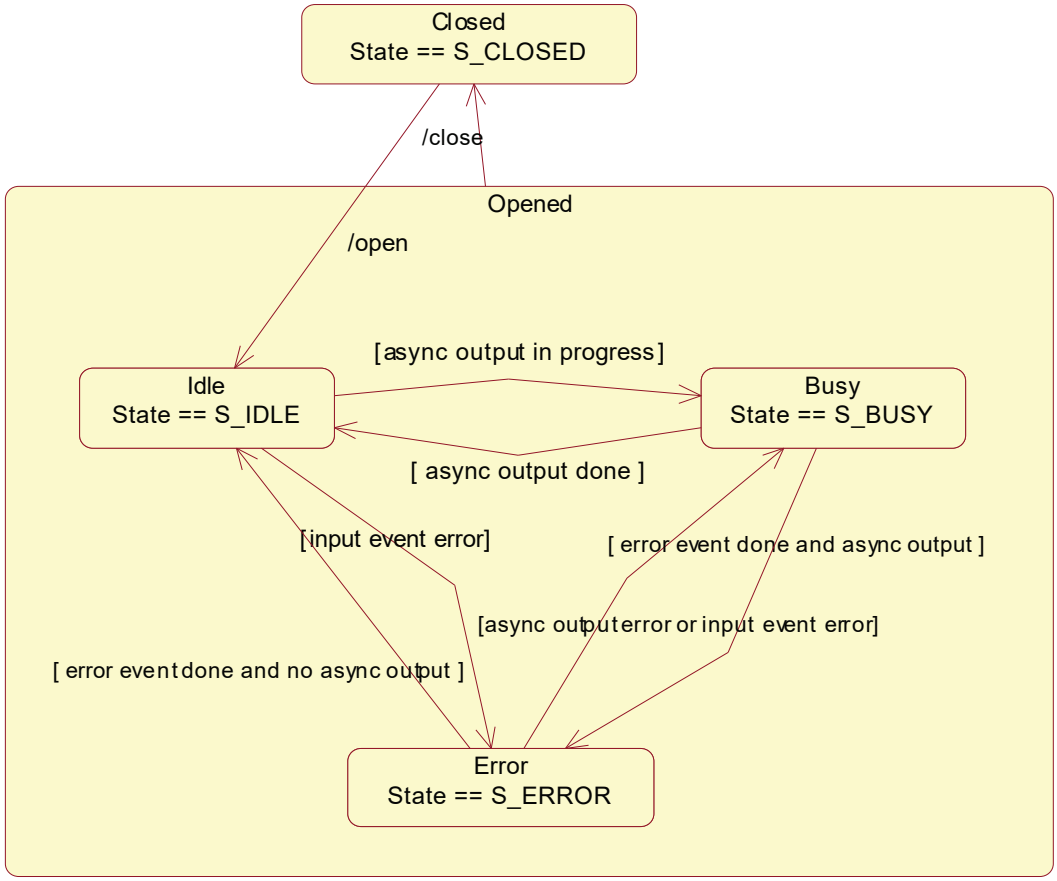
S_CLOSED
S_IDLE
S_BUSY
S_ERROR

The **State** property is set as follows:

- **State** is initially S_CLOSED.
- **State** is changed to S_IDLE when the **open** method is successfully called.
- **State** is set to S_BUSY when the Service is processing output. The **State** is restored to S_IDLE when the output has completed.
- The **State** is changed to S_ERROR when an asynchronous output encounters an error condition, or when an error is encountered during the gathering or processing of event-driven input.

  After the Service changes the **State** property to S_ERROR, it notifies the application of this error. The properties of this event are the error code and extended error code, the locus of the error, and a mutable response to the error.

1.3.6.15 Device State Diagram



1.3.6.16 Version Handling

As UnifiedPOS evolves, additional releases will introduce enhanced versions of some Devices. UnifiedPOS imposes the following requirements on Control and Service versions:

- **Control requirements.** A Control for a device category must operate with any Service for that category, as long as its major version number matches the Service's major version number. If they match, but the Control's minor version number is greater than the Service's minor version number, then the Control may support some new methods or properties that are not supported by the Service's release. If an application calls one of these methods or accesses one of these properties, the application will be notified of an error condition (E_NO_SERVICE).
- **Service requirements.** A Service for a device category must operate with any Control for that category, as long as its major version number matches the Control's major version number. If they match, but the Service's minor version number is greater than the Control's minor version number, then the Service may support some methods or properties that cannot be accessed from the Control.

When an application wishes to take advantage of the enhancements of a version, it must first determine that the Control and Service are at the proper major version and at or greater than the proper minor version. The versions are reported by the properties **DeviceControlVersion** and **DeviceServiceVersion**.

1.3.6.17 Deprecation Handling

Added in Release 1.11

In order to be able to rectify misunderstandings and/or ambiguities in the specification, a method of deprecation is required in order to eliminate these items over time.

Deprecation can be applied to Properties and Methods, as well as parameters, constants, and enumerations.

When an element is marked as deprecated, then Service providers are required to support the element's functionality for the following two **minor** releases of the standard. Starting with the third release of the standard after an element has been marked as deprecated, usage of the element will result in an E_DEPRECATED status.

When an element is marked as deprecated, then support for the element will be **removed** from the standard in the next **major** release of the standard after it is marked as deprecated.

All deprecated elements and the related versions when they were first marked as deprecated are listed in Annex H, Deprecation History on page H-1.

1.3.6.18 Hydra Device Considerations

Updated in Release 1.12

Initial Connectivity Model

When the development of the POS peripheral standard began, it was decided that the most flexible methodology would be to have an application be able to communicate to a peripheral through a two-layer process. Since the Microsoft's COM platform was the first supported architecture, Control Object and Service Object names were chosen. Later when Java was defined and the technology used precluded the use of "objects" as defined in the Windows world, the names were closely linked using the terminology Device Control and Device Service. Functionality however at the higher, abstracted level, remained the same.

Control Object or Device Control (Control)

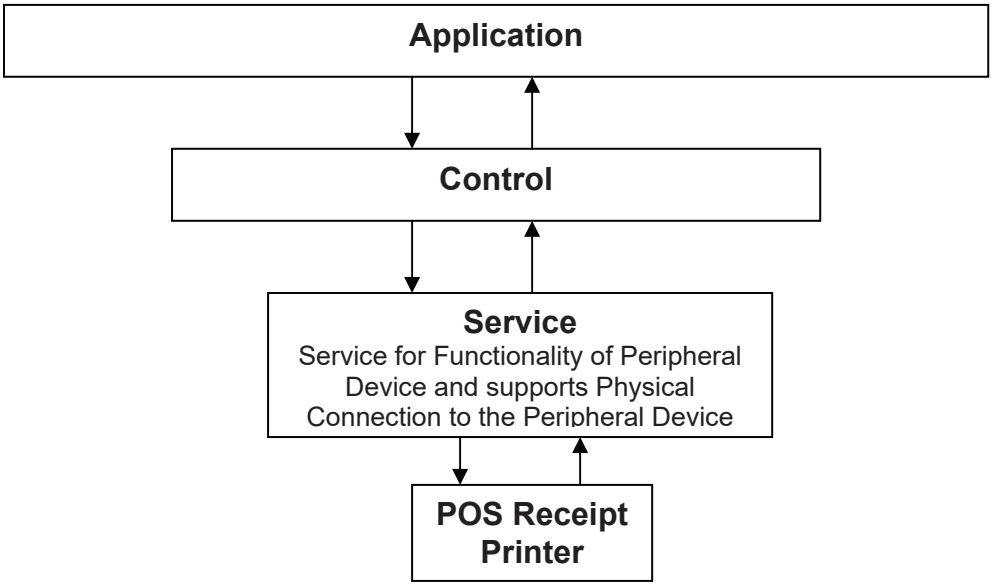
A thin layer of software was defined that would allow for what is commonly called “connecting the pipes” wherein a communication port would be opened and a device name would be assigned so that the application is able to communicate to the peripheral using that device name.

Service Object or Device Service (Service)

This incorporates usually vendor-specific code that interfaces with the peripheral device to allow for accessing, monitoring, processing, all the functionality of the peripheral device and exposing it to a common set of properties, methods, and events that an application needs to interact with the peripheral.

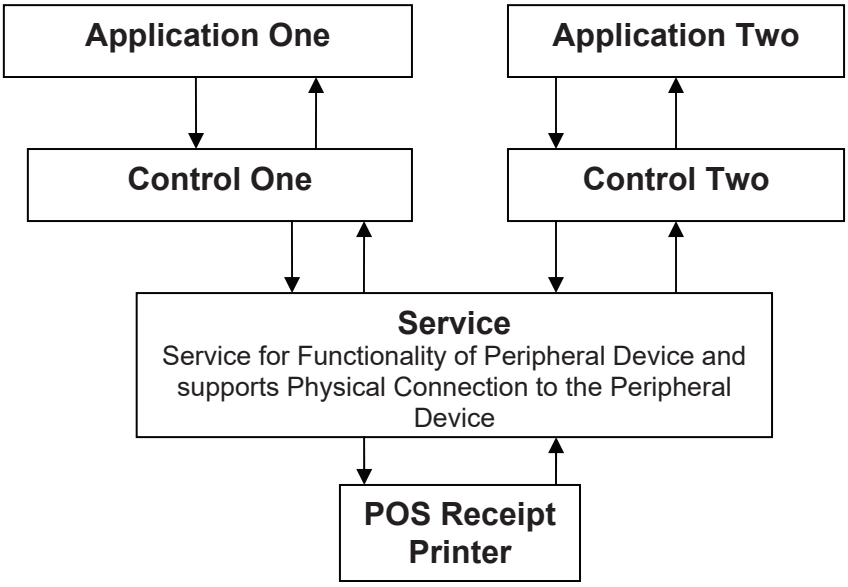
For mono-function peripheral devices, the process is very straightforward. In the most simplistic system one instance of a **Control** is instantiated to connect to the **Service**. As example for a simple POSPrinter:

Note: only one physical connection port (RS-232 for example) is used in this example.



Keeping things simple but adding another level of complexity is the case when more than one application needs to use the device. In this case, another **Control** is instantiated to the peripheral **Service** and all applications need to recognize that the peripheral is capable of being shared (for this example, assuming a shareable device) and utilize the **claim** and **release** methodology that the standard provides. In the POSPrinter example, this would look like…

***Note**: only one physical connection port (RS-232 for example) is used in this example.*



Note, that as far as each application is concerned, it is connected to the peripheral device and only one physical connection to the device is required... via the RS-232 serial connection in this example. This served the needs of device sharing where cooperating applications were utilized.

1.3.6.19 Multi-Function (Hydra) Peripheral Devices

The model needed to be expanded to cover the peripherals that include multiple device class functionality in a single unit. An example of such a device is a POS printer that may have additional functionality of being able to control a Customer Line Display, Cash Drawer, MICR, or other devices. These peripherals are referred to as “Hydra” peripherals alluding to the Greek mythology of a multi-headed animal that was connected to a single body interface.

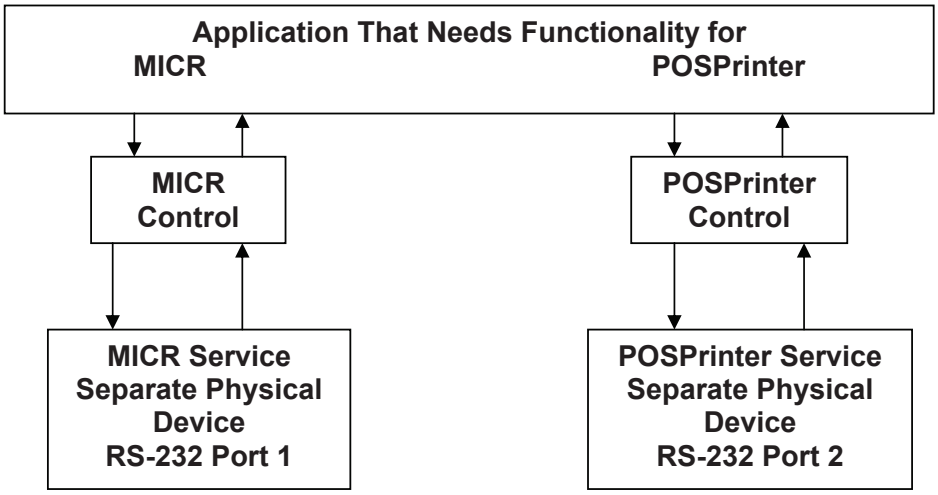
In the interaction of POS peripherals, the interface to the Application needs to be agnostic in its knowledge in either of the following cases…one where multiple physical peripheral devices are used or the other where one physical peripheral device incorporates the functionality of multiple physical peripheral devices.

Where multiple physical peripheral devices are present, multiple “pipes” (RS-232 serial ports for instance) are required…one for each of the physical peripheral devices.

In a Hydra peripheral only one “pipe” is required and it is used to communicate with all the various Device peripheral functionality of the connected peripheral device.

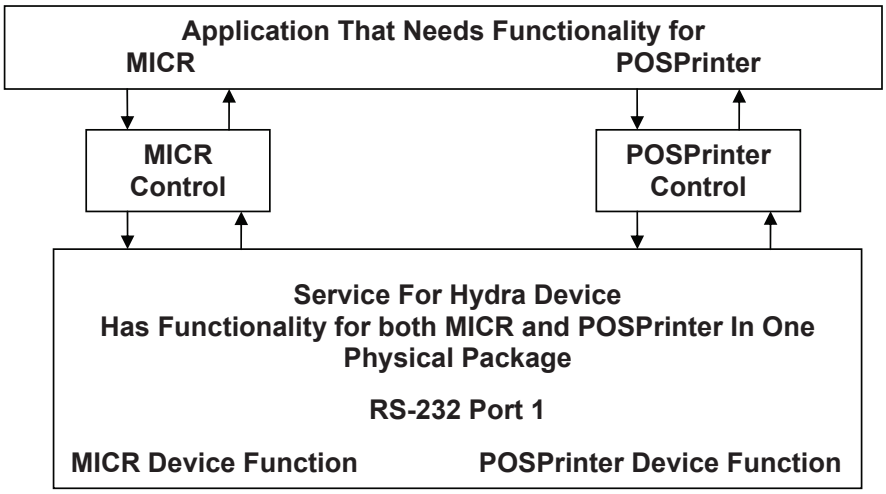
For example, consider the cases where in one instance a separate POSPrinter device and a separate MICR device is present; in another instance, a Hydra POSPrinter that has an incorporated MICR reader. The “look” to the Application(s) has to be agnostic…it should not care nor should it have to know which type of hardware device(s) are physically present. Ideally it should be able to use the same Application code to interact with either of the two implementations. For example:

***Note**: Application interfacing with two distinct peripherals.*



Note that in this case the application running the MICR and the POSPrinter consumes two separate ports but as far as the Application is concerned it interfaces to the MICR and POSPrinter functionality without regard to the fact that the two ports are used.

Note: Application interfacing with a Hydra peripheral.



Note that in this case the application running the MICR and POSPrinter consumes only one port but as far as the application is concerned it interfaces to the MICR and POSPrinter functionality without regard to the fact that only one port is used. It is up to the Hydra Service to control the port and route the functionality to and from the proper interface.

Considerations

While the desire is to have both interconnection techniques work the same with regards to the Application interface, problems do arise. In the Hydra case, an error state in one of the specific device functions may block the usage of the other function. This would not happen in the non-Hydra case since each peripheral is truly separate.

In our Printer and MICR Hydra case, the printer running out of paper might present a condition that would prevent reading a MICR code for instance. An error condition of “Out of Paper” would be reported through the POSPrinter interface but would not have any meaning to a route through the MICR interface. The Application requesting a MICR read in the Hydra case would be presented with an error or status condition that it would not get in the discrete MICR peripheral case. This presents a potential “hang up” condition or unresolved error situation.

Obviously an error condition needs to be reported to the application that is using the MICR functionality to alert it of a problem and allow for resolution. Rather than reporting a meaningless error of “Out of Paper” to the MICR application, a general **E_FAILURE** error would be sent back to the MICR application to alert it of the problem. The MICR application would then be responsible to go through an error recovery procedure to rectify the situation. It would go through an error recover operation that would present a console message informing the operator of an impending problem with usage of the MICR device.

Operator knowledge of the specific device would then be used to correct the problem. In this case knowing that the MICR is part of the printer would focus the attention of the Operator to the “Paper Out” status indicator. The resolution would be to replace the paper which would then clear the error condition for the MICR as well as the Printer.

Notice that every attempt is made to make the interaction with the peripheral device or Hydra peripheral device “look the same” to the application. Careful **Service** design needs to be used to make sure this is accomplished. Device vendors should define any limitations and unusual error conditions that may exist when accessing such hydra devices in their user documentation. Application developers should be aware of the possibility of discrete and Hydra POS devices when crafting their software and plan their error resolution accordingly.

2 Common Properties, Methods, and Events

2.1 General

The following Properties, Methods, and Events are used for all device categories unless noted otherwise in the *Usage Notes* table entry. For an overview of the general rules and guidelines, see "Device Behavior Models" on page 10.

2.2 Summary

Updated in Release 1.10

The following property list is a summary of the JavaPOS Common Properties. This list is used throughout the main UnifiedPOS chapters. Further details may be found in Annex B.

The OPOS implementation adds the following Common Properties:

BinaryConversion, OpenResult, ResultCode, and ResultCodeExtended.

Also, the last six properties are replaced by:

ControlObjectDescription, ControlObjectVersion, ServiceObjectDescription, ServiceObjectVersion, DeviceDescription, and DeviceName.

Further details may be found in Annex A.

Properties (UML attributes)

Name	*Type*	*Mutability*	*Version*	*Usage Notes*
AutoDisable:	*boolean*	{ read-write }	1.2	*1*
CapCompareFirmwareVersion:	*boolean*	{ read-only }	1.9	
CapPowerReporting:	*int32*	{ read-only }	1.3	
CapStatisticsReporting:	*boolean*	{ read-only }	1.8	
CapUpdateFirmware:	*boolean*	{ read-only }	1.9	
CapUpdateStatistics:	*boolean*	{ read-only }	1.8	
CheckHealthText:	*string*	{ read-only }	1.0	
Claimed:	*boolean*	{ read-only }	1.0	
DataCount:	*int32*	{ read-only }	1.2	*1*
DataEventEnabled:	*boolean*	{ read-write }	1.0	*1*
DeviceEnabled:	*boolean*	{ read-write }	1.0	
FreezeEvents:	*boolean*	{ read-write }	1.0	
OutputID:	*int32*	{ read-only }	1.0	*2*
PowerNotify:	*int32*	{ read-write }	1.3	
PowerState:	*int32*	{ read-only }	1.3	
State:	*int32*	{ read-only }	1.0	

DeviceControlDescription:	*string*	{ read-only }	1.0
DeviceControlVersion:	*int32*	{ read-only }	1.0
DeviceServiceDescription:	*string*	{ read-only }	1.0
DeviceServiceVersion:	*int32*	{ read-only }	1.0
PhysicalDeviceDescription:	*string*	{ read-only }	1.0
PhysicalDeviceName:	*string*	{ read-only }	1.0

Usage Notes:

1. Used only with Devices that have Event Driven Input.
2. Used only with Asynchronous Output Devices.

Methods (UML operations)

Name	*Version*
open (logicalDeviceName: *string* **):** **void { raises-exception }**	1.0
close ( ): **void { raises-exception }**	1.0
claim[a] (timeout: *int32* **):** **void { raises-exception }**	1.0
release[a] ( ): **void { raises-exception }**	1.0
checkHealth (level: *int32* **):** **void { raises-exception }**	1.0
clearInput ( ): **void { raises-exception }**	1.0
clearInputProperties ( ): **void { raises-exception }**	**1.10**
clearOutput ( ): **void { raises-exception }**	1.0
directIO (command: *int32*, **inout data:** *int32*, **inout obj:** *object* **):** **void { raises-exception }**	1.0
compareFirmwareVersion (firmwareFileName: *string*, **out result:** *int32* **):** **void { raises-exception }**	1.9
resetStatistics (statisticsBuffer: *string* **):** **void { raises-exception }**	1.8
retrieveStatistics (inout statisticsBuffer: *string* **):** **void { raises-exception }**	1.8
updateFirmware (firmwareFileName: *string* **):** **void { raises-exception }**	1.9
updateStatistics (statisticsBuffer: *string* **):** **void { raises-exception }**	1.8

a. **Note:** In the OPOS environment starting with Release 1.5, the **Claim** and **Release** methods are also defined as **ClaimDevice** and **ReleaseDevice** respectively due to **Release** being a reserved method used by Microsoft's Component Object Model (COM).

Events (UML interfaces)

Name	*Type*	*Mutability*	*Version*	*Usage Notes*
upos::events::DataEvent			1.0	*1*
Status:	*int32*	{ read-only }		
upos::events::DirectIOEvent			1.0	
EventNumber:	*int32*	{ read-only }		
Data:	*int32*	{ read-write }		
Obj:	*object*	{ read-write }		
upos::events::ErrorEvent			1.0	
ErrorCode:	*int32*	{ read-only }		
ErrorCodeExtended:	*int32*	{ read-only }		
ErrorLocus:	*int32*	{ read-only }		
ErrorResponse:	*int32*	{ read-write }		
upos::events::OutputCompleteEvent			1.0	*2*
OutputID:	*int32*	{ read-only }		
upos::events::StatusUpdateEvent			1.0	
Status:	*int32*	{ read-only }		

Usage Notes:

1. Used only with Devices that have Event Driven Input.
2. Used only with Asynchronous Output Devices.

2.3 General Information

This section lists properties, methods, and events that are common to many of the peripheral devices covered in this standard.

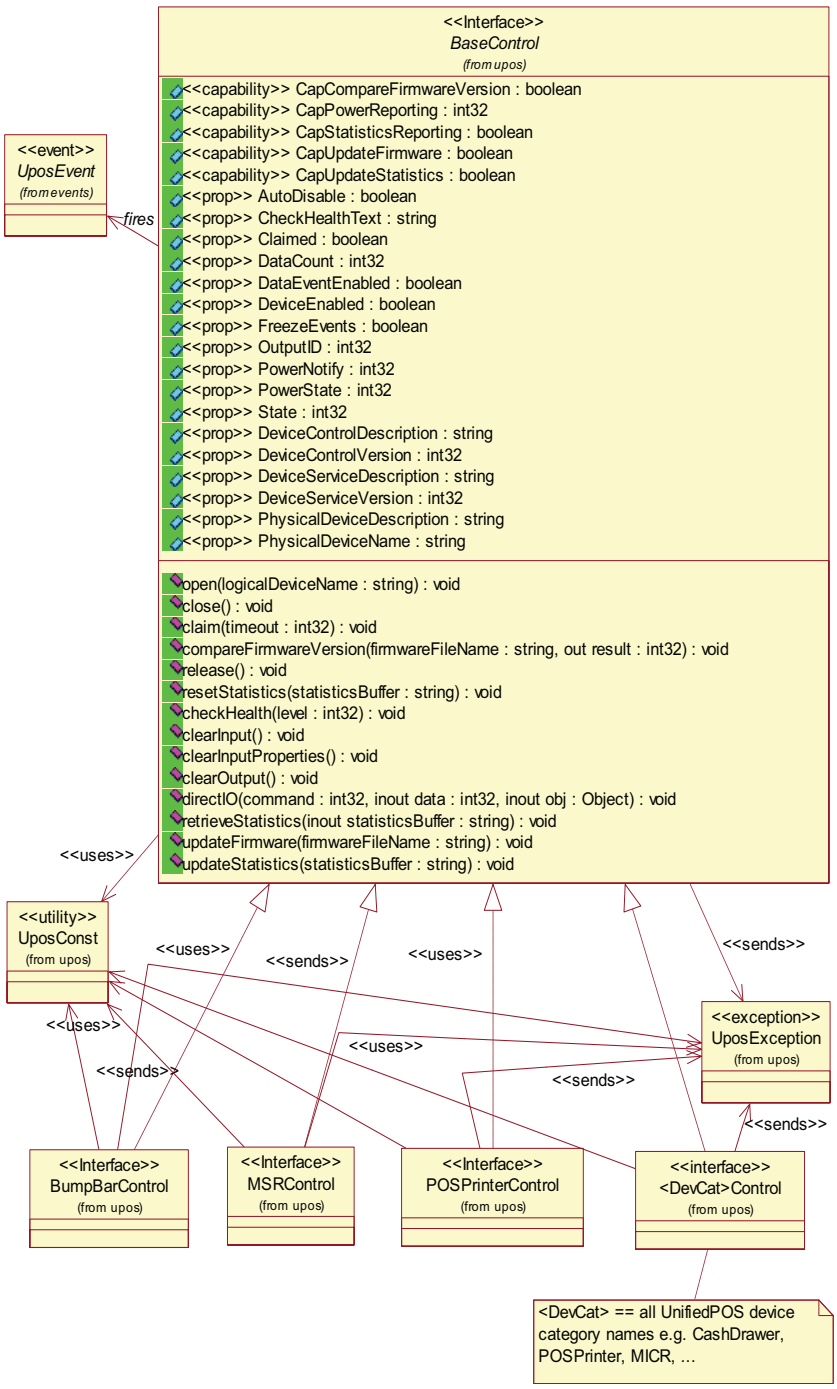
The summary section of each device category marks those common properties, methods, and events that do not apply to that category as "Not Supported." Items identified in this fashion are not present in the Control's class.

A good understanding of the features of the UnifiedPOS architecture model is required. Please see "Device Behavior Models" on page 10 for additional information.

2.3.1 Common PME Class Diagram

Updated in Release 1.10

The following diagram shows the relationships between the Common classes.



Notes: **AutoDisable**, **DataCount**, and **DataEventEnabled** are used only with Devices that have Event Driven Input.

OutputID is used only with Asynchronous Output Devices.

2.4 Properties (UML attributes)

2.4.1 AutoDisable Property

Syntax **AutoDisable:** ***boolean*** **{ read-write }**

Remarks If true, the UnifiedPOS Service will set **DeviceEnabled** to false after it receives and enqueues data as a **DataEvent**. Before any additional input can be received, the application must set **DeviceEnabled** to true.

If false, the UnifiedPOS Service does not automatically disable the device when data is received.

This property provides the application with an additional option for controlling the receipt of input data. If an application wants to receive and process only one input, or only one input at a time, then this property should be set to true. This property applies only to event-driven input devices.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Input Model" on page 18.

2.4.2 CapCompareFirmwareVersion Property

Revised in Release 1.14

Syntax **CapCompareFirmwareVersion:** ***boolean*** **{ read-only, access after open }**

Remarks If true, then the Service/device supports comparing the version of the firmware in the physical device against that of a firmware file; initialized by **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **compareFirmwareVersion** Method.

2.4.3 CapPowerReporting Property

Updated in Release 1.11

Syntax **CapPowerReporting:** ***int32*** **{ read-only }**

Remarks Identifies the reporting capabilities of the Device. It has one of the following values:

Value	Meaning
PR_NONE	The UnifiedPOS Service cannot determine the state of the device. Therefore, no power reporting is possible.
PR_STANDARD	The UnifiedPOS Service can determine and report two of the power states - OFF_OFFLINE (that is, off or offline) and ONLINE.
PR_ADVANCED	The UnifiedPOS Service can determine and report all three power states - OFF, OFFLINE, and ONLINE.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Power Reporting Model" on page 22, **PowerState** Property, **PowerNotify** Property.

2.4.4 CapStatisticsReporting Property

Added in Release 1.8

Syntax **CapStatisticsReporting:** ***boolean*** **{ read-only }**

Remarks If true, the device accumulates and can provide various statistics regarding usage; otherwise no usage statistics are accumulated. The information accumulated and reported is device specific, and is retrieved using the **retrieveStatistics** method.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **retrieveStatistics** Method.

2.4.5 CapUpdateFirmware Property

Updated in Release 1.14

Syntax **CapUpdateFirmware:** ***boolean*** **{ read-only, access after open }**

Remarks If true, then the device's firmware can be updated via the **updateFirmware** method; initialized by **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **updateFirmware** Method.

2.4.6 CapUpdateStatistics Property

Added in Release 1.8

Syntax **CapUpdateStatistics:** ***boolean*** **{ read-only }**

Remarks If true, the device statistics, or some of the statistics, can be reset to zero using the **resetStatistics** method, or updated using the **updateStatistics** method.

If **CapStatisticsReporting** is false, then **CapUpdateStatistics** is also false.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **CapStatisticsReporting** Property, **resetStatistics** Method, **updateStatistics** Method.

2.4.7 CheckHealthText Property

Syntax **CheckHealthText:** ***string*** **{ read-only }**

Remarks Holds the results of the most recent call to the **checkHealth** method. The following examples illustrate some possible diagnoses:

- "Internal HCheck: Successful"
- "External HCheck: Not Responding"
- "Interactive HCheck: Complete"

This property is empty ("") before the first call to the **checkHealth** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **checkHealth** Method.

2.4.8 Claimed Property

Syntax **Claimed:** ***boolean*** **{ read-only }**

Remarks If true, the device is claimed for exclusive access. If false, the device is released for sharing with other applications.

Many devices must be claimed before the Control will allow access to many of its methods and properties, and before it will deliver events to the application.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Initialization and Finalization" on page 12, "Device Sharing Model" on page 14, **claim** Method, **release** Method.

2.4.9 DataCount Property

Syntax **DataCount:** ***int32*** **{ read-only }**

Remarks Holds the number of enqueued **DataEvent**s.

The application may read this property to determine whether additional input is enqueued from a device, but has not yet been delivered because of other application processing, freezing of events, or other causes.

This property is initialized to zero by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Input Model" on page 18, **DataEvent**.

2.4.10 DataEventEnabled Property

Syntax **DataEventEnabled:** ***boolean*** **{ read-write }**

Remarks If true, a **DataEvent** will be delivered as soon as input data is enqueued. If changed to true and some input data is already queued, then a **DataEvent** is delivered immediately. (Note that other conditions may delay "immediate" delivery: if **FreezeEvents** is true or another event is already being processed at the application, the **DataEvent** will remain queued at the UnifiedPOS Service until the condition is corrected.)

If false, input data is enqueued for later delivery to the application. Also, if an input error occurs, the **ErrorEvent** is not delivered while this property is false.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Events" on page 15, **DataEvent**.

2.4.11 DeviceControlDescription Property

Syntax **DeviceControlDescription:** ***string*** **{ read-only }**

Remarks Holds an identifier for the UnifiedPOS Control and the company that produced it.

A sample returned string is:

```
"POS Printer UnifiedPOS Compatible Control, (C) 1998 Epson"
```

This property is always readable.

Errors None.

See Also **DeviceControlVersion** Property.

2.4.12 DeviceControlVersion Property

Syntax **DeviceControlVersion:** ***int32*** **{ read-only }**

Remarks Holds the UnifiedPOS Control version number.

Three version levels are specified, as follows:

Version Level	Description
Major	The “millions” place. A change to the UnifiedPOS major version level for a device class reflects significant interface enhancements, and may remove support for obsolete interfaces from previous major version levels.
Minor	The “thousands” place. A change to the UnifiedPOS minor version level for a device class reflects minor interface enhancements, and must provide a superset of previous interfaces at this major version level.
Build	The “units” place. Internal level provided by the UnifiedPOS Control developer. Updated when corrections are made to the UnifiedPOS Control implementation.

A sample version number is:

```
1002038
```

This value may be displayed as version “1.2.38”, and interpreted as major version 1, minor version 2, build 38 of the UnifiedPOS Control.

This property is always readable.

Errors None.

See Also “Version Handling” on page 29, **DeviceControlDescription** Property.

2.4.13 DeviceEnabled Property

Syntax **DeviceEnabled:** ***boolean*** **{ read-write }**

Remarks If true, the device is in an operational state. If changed to true, then the device is brought to an operational state.

If false, the device has been disabled. If changed to false, then the device is physically disabled when possible, any subsequent input will be discarded, and output operations are disallowed.

Changing this property usually does not physically affect output devices. For consistency, however, the application must set this property to true before using output devices.

The Device’s power state may be reported while **DeviceEnabled** is true; See “Device Power Reporting Model” on page 22 for details.

This property is initialized to false by the **open** method. Note that an exclusive use device must be claimed before the device may be enabled.

Errors A UposException may be thrown when this property is accessed. For further information, see “Errors” on page 16.

See Also “Device Initialization and Finalization” on page 12.

2.4.14 DeviceServiceDescription Property

Syntax **DeviceServiceDescription:** ***string*** **{ read-only }**

Remarks Holds an identifier for the UnifiedPOS Service and the company that produced it.

A sample returned string is:

```
"TM-U950 Printer UnifiedPOS Compatible Service Driver, (C) 1998
Epson"
```

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

2.4.15 DeviceServiceVersion Property

Syntax **DeviceServiceVersion:** ***int32*** **{ read-only }**

Remarks Holds the UnifiedPOS Service version number.

Three version levels are specified, as follows:

Version Level	Description
Major	The "millions" place. A change to the UnifiedPOS major version level for a device class reflects significant interface enhancements, and may remove support for obsolete interfaces from previous major version levels.
Minor	The "thousands" place. A change to the UnifiedPOS minor version level for a device class reflects minor interface enhancements, and must provide a superset of previous interfaces at this major version level.
Build	The "units" place. Internal level provided by the UnifiedPOS Service developer. Updated when corrections are made to the UnifiedPOS Service implementation.

A sample version number is:

```
1002038
```

This value may be displayed as version "1.2.38", and interpreted as major version 1, minor version 2, build 38 of the UnifiedPOS Service.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Version Handling" on page 29, **DeviceServiceDescription** Property.

2.4.16 FreezeEvents Property

Updated in Release 1.12

Syntax **FreezeEvents:** ***boolean*** **{ read-write }**

Remarks If true, the UnifiedPOS Control will not deliver events. Events will be enqueued until this property is set to false.

If false, the application allows events to be delivered. If some events have been held while events were frozen and all other conditions are correct for delivering the events, then changing this property to false will allow these events to be delivered. An application may choose to freeze events for a specific sequence of code where interruption by an event is not desirable.

Unless specified otherwise, properties that convey device state information (e.g., **JrnEmpty** and **DrawerOpened**) are kept current while the device is enabled, regardless of the setting of this property.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

2.4.17 OutputID Property

Syntax **OutputID:** ***int32*** **{ read-only }**

Remarks Holds the identifier of the most recently started asynchronous output.

When a method successfully initiates an asynchronous output, the Device assigns an identifier to the request. When the output completes, an **OutputCompleteEvent** will be enqueued with this output ID as a parameter.

The output ID numbers are assigned by the UnifiedPOS Service and are guaranteed to be unique among the set of outstanding asynchronous outputs. No other facts about the ID should be assumed.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Output Models" on page 21, **OutputCompleteEvent**.

2.4.18 PowerNotify Property

Syntax **PowerNotify:** ***int32*** **{ read-write }**

Remarks Contains the type of power notification selection made by the Application. It has one of the following values:

Value	Meaning
PN_DISABLED	The UnifiedPOS Service will not provide any power notifications to the application. No power notification **StatusUpdateEvent**s will be fired, and **PowerState** may not be set.
PN_ENABLED	The UnifiedPOS Service will fire power notification **StatusUpdateEvent**s and update **PowerState**, beginning when **DeviceEnabled** is set to true. The level of functionality depends upon **CapPowerReporting**.

PowerNotify may only be set while the device is disabled; that is, while **DeviceEnabled** is false.

This property is initialized to PN_DISABLED by the **open** method. This value provides compatibility with earlier releases.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	One of the following occurred: • The device is already enabled. • **PowerNotify** = PN_ENABLED but **CapPowerReporting** = PR_NONE.

See Also "Device Power Reporting Model" on page 22, **CapPowerReporting** Property, **PowerState** Property.

2.4.19 PowerState Property

Updated in Release 1.11

Syntax **PowerState:** ***int32*** **{ read-only }**

Remarks Identifies the current power condition of the device, if it can be determined.
It has one of the following values:

Value	Meaning
PS_UNKNOWN	Cannot determine the device's power state for one of the following reasons: **CapPowerReporting** = PR_NONE; the device does not support power reporting. **PowerNotify** = PN_DISABLED; power notifications are disabled. **DeviceEnabled** = false; Power state monitoring does not occur until the device is enabled.
PS_ONLINE	The device is powered on and ready for use. Can be returned if **CapPowerReporting** = PR_STANDARD or PR_ADVANCED.
PS_OFF	The device is powered off or detached from the POS terminal. Can only be returned if **CapPowerReporting** = PR_ADVANCED.
PS_OFFLINE	The device is powered on but is either not ready or not able to respond to requests. Can only be returned if **CapPowerReporting** = PR_ADVANCED.
PS_OFF_OFFLINE	The device is either off or off-line. Can only be returned if **CapPowerReporting** = PR_STANDARD.

This property is initialized to PS_UNKNOWN by the **open** method. When **PowerNotify** is set to enabled and **DeviceEnabled** is true, then this property is updated as the UnifiedPOS Service detects power condition changes.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also "Device Power Reporting Model" on page 22, **CapPowerReporting** Property, **PowerNotify** Property.

2.4.20 PhysicalDeviceDescription Property

Syntax **PhysicalDeviceDescription:** ***string*** **{ read-only }**

Remarks Holds an identifier for the physical device.

A sample returned string is:

```
"NCR 7192-0184 Printer, Japanese Version"
```

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **PhysicalDeviceName** Property.

2.4.21 PhysicalDeviceName Property

Syntax **PhysicalDeviceName:** ***string*** **{ read-only }**

Remarks Holds a short name identifying the physical device. This is a short version of **PhysicalDeviceDescription** and should be limited to 30 characters.

This property will typically be used to identify the device in an application message box, where the full description is too verbose. A sample returned string is:

```
"IBM Model II Printer, Japanese"
```

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see "Errors" on page 16.

See Also **PhysicalDeviceDescription** Property.

2.4.22 State Property

Syntax **State:** ***int32*** **{ read-only }**

Remarks Holds the current state of the Device. It has one of the following values:

Value	Meaning
S_CLOSED	The Device is closed.
S_IDLE	The Device is in a good state and is not busy.
S_BUSY	The Device is in a good state and is busy performing output.
S_ERROR	An error has been reported, and the application must recover the Device to a good state before normal I/O can resume.

This property is always readable.

Errors None.

See Also "Device Information Reporting Model" on page 25.

2.5 Methods (UML operations)

2.5.1 checkHealth Method

Syntax **checkHealth (level:** ***int32*** **):**
void { raises-exception }

The *level* parameter indicates the type of health check to be performed on the device. The following values may be specified:

Value	Meaning
CH_INTERNAL	Perform a health check that does not physically change the device. The device is tested by internal tests to the extent possible.
CH_EXTERNAL	Perform a more thorough test that may change the device. For example, a pattern may be printed on the printer.
CH_INTERACTIVE	Perform an interactive test of the device. The supporting UnifiedPOS Service will typically display a modal dialog box to present test options and results.

Remarks Tests the state of a device.

A text description of the results of this method is placed in the **CheckHealthText** property. The health of many devices can only be determined by a visual inspection of these test results.

This method is always synchronous.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The specified health check level is not supported by the UnifiedPOS Service.

See Also **CheckHealthText** Property.

2.5.2 claim Method

Updated in Release 1.11

Syntax **claim (timeout: *int32*):**
void { raises-exception }

The *timeout* parameter gives the maximum number of milliseconds to wait for exclusive access to be satisfied. If zero, then immediately either returns (if successful) or throws an appropriate exception. If FOREVER (-1), the method waits as long as needed until exclusive access is satisfied.

Remarks Requests exclusive access to the device. Many devices require an application to claim them before they can be used.

When successful, the **Claimed** property is changed to true.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	This device cannot be claimed for exclusive access, or an invalid timeout parameter was specified.
E_TIMEOUT	Another application has exclusive access to the device, and did not relinquish control before timeout milliseconds expired.

See Also "Device Initialization and Finalization" on page 12, "Device Sharing Model" on page 14, **release** Method.

2.5.3 clearInput Method

Syntax **clearInput ( ):**
void { raises-exception }

Remarks Clears all device input that has been buffered.

Any data events or input error events that are enqueued – usually waiting for **DataEventEnabled** to be set to true and **FreezeEvents** to be set to false – are also cleared.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

See Also "Device Input Model" on page 18.

2.5.4 clearInputProperties Method

Added in Release 1.10

Syntax **clearInputProperties ( ):**
void { raises-exception }

Remarks Sets all data properties that were populated as a result of firing a **DataEvent** or **ErrorEvent** back to their default values. This does not reset the **DataCount** or **State** properties.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

See Also "Device Input Model" on page 18.

2.5.5 clearOutput Method

Updated in Release 1.7

Syntax **clearOutput ( ):**
void { raises-exception }

Remarks Clears all buffered output data, including all asynchronous output. Also, when possible, halts outputs that are in progress.

Any output error events that are enqueued – usually waiting for **FreezeEvents** to be set to false – are also cleared.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

See Also "Device Output Models" on page 21.

2.5.6 close Method

Syntax

close ( ):
void { raises-exception }

Remarks Releases the device and its resources.

If the **DeviceEnabled** property is true, then the device is disabled.

If the **Claimed** property is true, then exclusive access to the device is released.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

See Also "Device Initialization and Finalization" on page 12, **open** Method.

2.5.7 compareFirmwareVersion Method

Added in Release 1.9

Syntax

compareFirmwareVersion (firmwareFileName: ***string*****, out result:** ***int32*** **):**
void { raises-exception, use after open-claim-enable }

Parameter	Description
firmwareFileName	Specifies either the name of the file containing the firmware or a file containing a set of firmware files whose versions are to be compared against those of the device.
result	Location in which to return the result of the comparison.

Remarks This method determines whether the version of the firmware contained in the specified file is newer than, older than, or the same as the version of the firmware in the physical device.

The Service should check that the specified firmware file exists and that its contents are valid for this device before attempting to perform the comparison operation.

The result of the comparison is returned in the *result* parameter and will be one of the following values:

Value	Meaning
CFV_FIRMWARE_OLDER	Indicates that the version of one or more of the firmware files is older than the firmware in the device and that none of the firmware files is newer than the firmware in the device.

CFV_FIRMWARE_SAME	Indicates that the versions of all of the firmware files are the same as the firmware in the device.
CFV_FIRMWARE_NEWER	Indicates that the version of one or more of the firmware files is newer than the firmware in the device and that none of the firmware files is older than the firmware in the device.
CFV_FIRMWARE_DIFFERENT	Indicates that the version of one or more of the firmware files is different than the firmware in the device, but either: • The chronological relationship cannot be determined, or • The relationship is inconsistent -- one or more are older while one or more are newer.
CFV_FIRMWARE_UNKNOWN	Indicates that a relationship between the two firmware versions could not be determined. A possible reason for this *result* could be an attempt to compare Japanese and US versions of firmware.

If the *firmwareFileName* parameter specifies a file list, all of the component firmware files should reside in the same directory as the firmware list file. This will allow for distribution of the updated firmware without requiring a modification to the firmware list file.

Errors

A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapCompareFirmwareVersion** is false.
E_NOEXIST	The file specified by *firmwareFileName* does not exist or, if *firmwareFileName* specifies a file list, one or more of the component firmware files are missing.
E_EXTENDED	*ErrorCodeExtended* = EFIRMWARE_BAD_FILE: The specified firmware file or files exist, but one or more are either not in the correct format or are corrupt.

See Also

CapCompareFirmwareVersion Property.

2.5.8 directIO Method

Syntax **directIO (command:** ***int32*****, inout data:** ***int32*****, inout obj:** ***object*** **):**
void { raises-exception }

Parameter	Description
command	Command number whose specific values are assigned by the UnifiedPOS Service.
data	An array of one mutable integer whose specific values or usage vary by *command* and UnifiedPOS Service.
obj	Additional data whose usage varies by *command* and UnifiedPOS Service.

Remarks Communicates directly with the UnifiedPOS Service.

This method provides a means for a UnifiedPOS Service to provide functionality to the application that is not otherwise supported by the standard UnifiedPOS Control for its device category. Depending upon the UnifiedPOS Service's definition of the command, this method may be asynchronous or synchronous.

Use of this method will make an application non-portable. The application may, however, maintain portability by performing **directIO** calls within conditional code. This code may be based upon the value of the **DeviceServiceDescription**, **PhysicalDeviceDescription**, or **PhysicalDeviceName** property.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

See Also **DirectIOEvent.**

2.5.9 open Method

Updated in Release 1.7

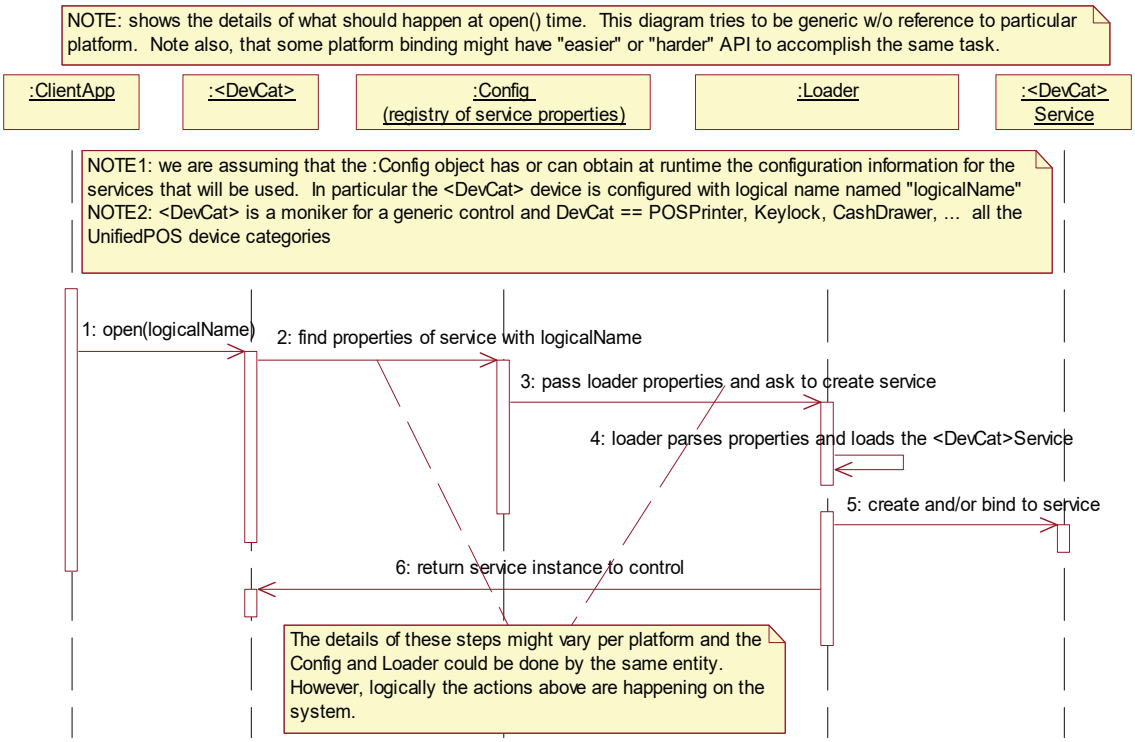
Syntax **open (logicalDeviceName:** ***string*** **):**
void { raises-exception }

The *logicalDeviceName* parameter specifies the device name to open.

Remarks Opens a device for subsequent I/O.

The device name specifies which of one or more devices supported by this UnifiedPOS Control should be used. The *logicalDeviceName* must exist in the operating system's reference locater system (such as the JavaPOS Configurator/Loader (JCL) or the Window's Registry) for this device category so that its relationship to the physical device can be determined. Entries in the reference locator's system are created by a setup or configuration utility.

The following sequence diagram shows the details of what needs to happen during the **open** method call processing to allow the creation of the Service and its binding to the Control.



When this method is successful, it initializes the properties **Claimed**, **DeviceEnabled**, **DataEventEnabled**, and **FreezeEvents**, as well as descriptions and version numbers of the UnifiedPOS software layers. Additional category-specific properties may also be initialized.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The UnifiedPOS Control is already open.
E_NOEXIST	The specified *logicalDeviceName* was not found.
E_NOSERVICE	Could not establish a connection to the corresponding UnifiedPOS Service.

See Also "Device Initialization and Finalization" on page 12, "Version Handling" on page 29, **close** Method.

2.5.10 release Method

Syntax **release ( ):**
void { raises-exception }

Remarks Releases exclusive access to the device.

If the **DeviceEnabled** property is true, and the device is an exclusive-use device, then the device is also disabled (this method does not change the device enabled state of sharable devices).

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The application does not have exclusive access to the device.

See Also "Device Sharing Model" on page 14, **claim** Method.

2.5.11 resetStatistics Method

Updated in Release 1.10

Syntax **resetStatistics (statisticsBuffer: *string*):**
void { raises-exception }

Parameter	Description
statisticsBuffer	The data buffer defining the statistics that are to be reset.

This is a comma-separated list of name(s), where an empty string ("") means ALL resettable statistics are to be reset, "U_" means all UnifiedPOS defined resettable statistics are to be reset, "M_" means all manufacturer defined resettable statistics are to be reset, and "actual_name1, actual_name2" (from the XML file definitions) means that the specifically defined resettable statistic(s) are to be reset.

Remarks Resets the defined resettable statistics in a device to zero. All the requested statistics must be successfully reset in order for this method to complete successfully, otherwise an *ErrorCode* of E_EXTENDED is returned.

Both **CapStatisticsReporting** and **CapUpdateStatistics** must be true in order to successfully use this method.

This method is always executed synchronously.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapStatisticsReporting** or **CapUpdateStatistics** is false, or the named statistic is not defined/resettable.
E_EXTENDED	*ErrorCodeExtended* = ESTATS_ERROR: At least one of the specified statistics could not be reset. *ErrorCodeExtended* = ESTATS_DEPENDENCY: At least one other statistic is required to be reset in addition to a requested statistic.

See Also **CapStatisticsReporting** Property, **CapUpdateStatistics** Property.

2.5.12 retrieveStatistics Method

Added in Release 1.8

Syntax **retrieveStatistics (inout statisticsBuffer:** ***string*** **):**
void { raises-exception }

Parameter	Description
statisticsBuffer	The data buffer defining the statistics to be retrieved and in which the retrieved statistics are placed.

This is a comma-separated list of name(s), where an empty string (“”) means ALL statistics are to be retrieved, “U_” means all UnifiedPOS defined statistics are to be retrieved, “M_” means all manufacturer defined statistics are to be retrieved, and “actual_name1, actual_name2” (from the XML file definitions) means that the specifically defined statistic(s) are to be retrieved.

Remarks Retrieves the requested statistics from a device.

CapStatisticsReporting must be true in order to successfully use this method.

This method is always executed synchronously.

All calls to **retrieveStatistics** will return the following XML as a minimum:

```
<?xml version='1.0' ?>
<UPOSStat version="1.13.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.omg.org/UnifiedPOS/namespace/"
xsi:schemaLocation="http://www.omg.org/UnifiedPOS/namespace/UPOSStat.xsd">
        <Event>
          <Parameter>
            <Name>RequestedStatistic</Name>
            <Value>1234</Value>
          </Parameter>
        </Event>
        <Equipment>
      <UnifiedPOSVersion>1.13</UnifiedPOSVersion>
      <DeviceCategory UPOS="CashDrawer"/>
      <ManufacturerName>Cashdrawers R Us</ManufacturerName>
      <ModelName>CD-123</ModelName>
      <SerialNumber>12345</SerialNumber>
      <FirmwareRevision>1.0 Rev. B</FirmwareRevision>
```

```
        <Interface>RS232</Interface>
        <InstallationDate>2000-03-01</InstallationDate>
    </Equipment>
</UPOSStat>
```

If the application requests a statistic name that the device does not support, the `<Parameter>` entry will be returned with an empty `<Value>`. e.g.,

```
<Parameter>
   <Name>RequestedStatistic</Name>
   <Value></Value>
</Parameter>
```

All statistics that the device collects that are manufacturer specific (not defined in the schema) will be returned in a `<ManufacturerSpecific>` tag instead of a `<Parameter>` tag. e.g.,

```
<ManufacturerSpecific>
   <Name>TheAnswer</Name>
   <Value>42</Value>
</ManufacturerSpecific>
```

When an application requests all statistics from the device, the device will return a `<Parameter>` entry for every defined statistic for the device category as defined by the XML schema version specified by the version attribute in the `<UPOSStat>` tag. If the device does not record any of the statistics, the `<Value>` tag will be empty.

The most up-to-date files defining the XML tag names and example schemas for the statistics for all device categories can be downloaded from the ARTS web site at http://retail.omg.org.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapStatisticsReporting** is false or the named statistic is not defined.

See Also **CapStatisticsReporting** Property.

2.5.13 updateFirmware Method

Added in Release 1.9

Syntax **updateFirmware (firmwareFileName: *string*):**
void { raises-exception, use after open-claim-enable }

Parameter	Description
firmwareFileName	Specifies either the name of the file containing the firmware or a file containing a set of firmware files that are to be downloaded into the device.

Remarks

This method updates the firmware of a device with the version of the firmware contained or defined in the file specified by the *firmwareFileName* parameter regardless of whether that firmware's version is newer than, older than, or the same as the version of the firmware already in the device. If the *firmwareFileName* parameter specifies a file list, all of the component firmware files should reside in the same directory as the firmware list file. This will allow for distribution of the updated firmware without requiring a modification to the firmware list file.

When this method is invoked, the Service should check that the specified firmware file exists and that its contents are valid for this device. If so, this method should return immediately and the remainder of the update firmware process should continue asynchronously.

The Service should notify the application of the status of the update firmware process by firing **StatusUpdateEvent**s with values of SUE_UF_PROGRESS + an integer between 1 and 100 indicating the completion percentage of the update firmware process. For application convenience, the **StatusUpdateEvent** value SUE_UF_COMPLETE is defined to be the same value as SUE_UF_PROGRESS + 100.

For consistency, the update firmware process is complete after the new firmware has been downloaded into the physical device, any necessary physical device reset has completed, and the Service and the physical device have been returned to the state they were in before the update firmware process began.

For consistency, a Service must always fire at least one **StatusUpdateEvent** with an incomplete progress completion percentage (i.e., a percentage between 1 and 99), even if the device cannot physically report the progress of the update firmware process. If the update firmware process completes successfully, the Service must fire a **StatusUpdateEvent** with a progress of 100 or use the special constant SUE_UF_COMPLETE, which has the same value. These Service requirements allow applications using this method to be designed to always expect some level of progress notification.

If an error is detected during the asynchronous portion of a update firmware process, one of the following **StatusUpdateEvent**s will be fired:

Value	Meaning
SUE_UF_FAILED_DEV_OK	The update firmware process failed but the device is still operational.
SUE_UF_FAILED_DEV_UNRECOVERABLE	The update firmware process failed and the device is neither usable nor recoverable through software. The device requires service to be returned to an operational state.
SUE_UF_FAILED_DEV_NEEDS_FIRMWARE	The update firmware process failed and the device will not be operational until another attempt to update the firmware is successful.
SUE_UF_FAILED_DEV_UNKNOWN	The update firmware process failed and the device is in an indeterminate state.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapUpdateFirmware** is false.
E_NOEXIST	The file specified by *firmwareFileName* does not exist or, if *firmwareFileName* specifies a file list, one or more of the component firmware files are missing.
E_EXTENDED	*ErrorCodeExtended* = EFIRMWARE_BAD_FILE: The specified firmware file or files exist, but one or more are either not in the correct format or are corrupt.

See Also **CapUpdateFirmware** Property.

2.5.14 updateStatistics Method

Updated in Release 1.10

Syntax **updateStatistics (statisticsBuffer:** ***string*** **):**
void { raises-exception }

Parameter	Description
statisticsBuffer	The data buffer defining the statistics with values that are to be updated.

This is a comma-separated list of name-value pair(s), where an empty string name (""=value1") means ALL resettable statistics are to be set to the value "value1," "U_=value2" means all UnifiedPOS defined resettable statistics are to be set to the value "value2," "M_=value3" means all manufacturer defined resettable statistics are to be set to the value "value3," and "actual_name1=value4, actual_name2=value5" (from the XML file definitions) means that the specifically defined resettable statistic(s) are to be set to the specified value(s).

Remarks Updates the defined resettable statistics in a device. All the requested statistics must be successfully updated in order for this method to complete successfully, otherwise an *ErrorCode* of E_EXTENDED is returned.

Both **CapStatisticsReporting** and **CapUpdateStatistics** must be true in order to successfully use this method.

This method is always executed synchronously.

Errors A UposException may be thrown when this method is invoked. For further information, see "Errors" on page 16.

Some possible values of the exception's *ErrorCode* property are:

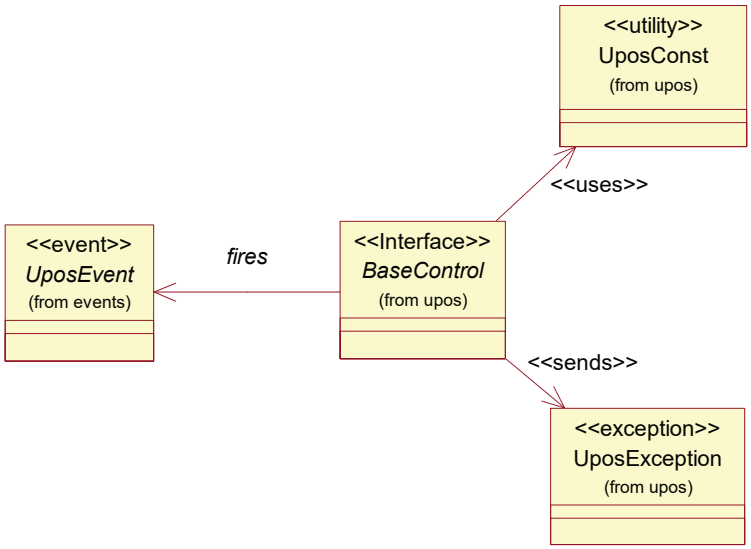
Value	Meaning
E_ILLEGAL	**CapStatisticsReporting** or **CapUpdateStatistics** is false, or the named statistic is not defined/updatable.
E_EXTENDED	*ErrorCodeExtended* = ESTATS_ERROR: At least one of the specified statistics could not be updated. *ErrorCodeExtended* = ESTATS_DEPENDENCY: At least one other statistic is required to be updated in addition to a requested statistic.

See Also **CapStatisticsReporting** Property, **CapUpdateStatistics** Property.

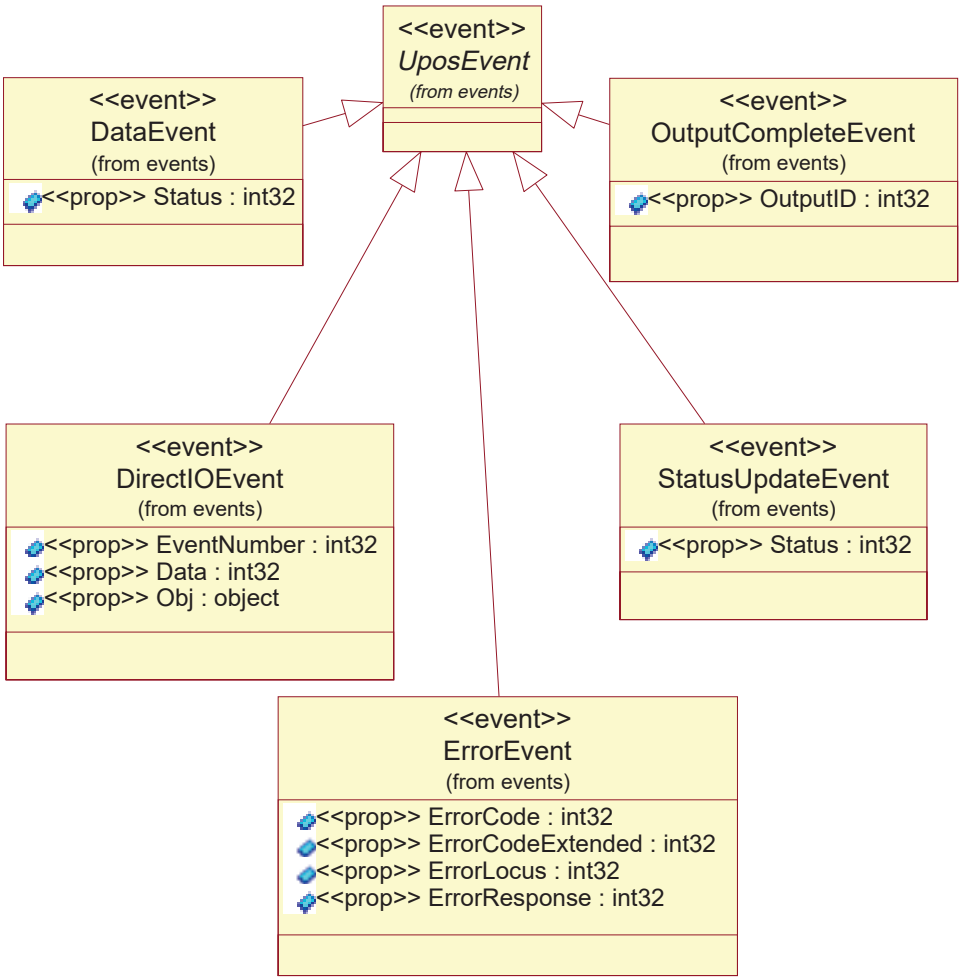
2.6 Events (UML interfaces)

The UnifiedPOS standard utilizes a common UML base control structure to derive a specific implementation case. The UML event base control model and interfaces are shown below for the events.

upos::BaseControl



upos::events interfaces



2.6.1 DataEvent

<<event>> **upos::events::DataEvent**
Status: ***int32*** **{ read-only }**

Description Notifies the application that input data is available from the device.

Attribute This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	The input status with its value dependent upon the device category; it may describe the type or qualities of the input data.

Remarks When this event is delivered to the application, the **DataEventEnabled** property is changed to false, so that no further data events will be delivered until the application sets **DataEventEnabled** back to true. The actual *byte array* input data is placed in one or more device-specific properties.

If **DataEventEnabled** is false at the time that data is received, then the data is enqueued in an internal buffer, the device-specific input data properties are not updated, and the event is not delivered. When **DataEventEnabled** is subsequently changed back to true, the event will be delivered immediately if input data is enqueued and **FreezeEvents** is false.

See Also "Errors" on page 16, "Device Input Model" on page 18, **DataEventEnabled** Property, **FreezeEvents** Property.

2.6.2 DirectIOEvent

Updated in Release 1.7

<<event>> **upos::events::DirectIOEvent**
EventNumber: ***int32*** **{ read-only }**
Data: ***int32*** **{ read-write }**
Obj: ***object*** **{ read-write }**

Description Provides UnifiedPOS Service information directly to the application. This event provides a means for a vendor-specific UnifiedPOS Service to provide events to the application that are not otherwise supported by the UnifiedPOS Control.

Attributes This event contains the following attributes:

Attribute	Type	Description
EventNumber	*int32*	Event number whose specific values are assigned by the UnifiedPOS Service.
Data	*int32*	Additional numeric data. Specific values vary by the *EventNumber* and the UnifiedPOS Service. This attribute is settable.
Obj	*object*	Additional data whose usage varies by the *EventNumber* and the UnifiedPOS Service. This attribute is settable.[1]

1. In the **OPOS** environment, the format of this data depends upon the value of the **BinaryConversion** property. See **BinaryConversion** property in Annex A.

Remarks This event is to be used only for those types of vendor specific functions that are not otherwise described as part of the UnifiedPOS standard. Use of this event may restrict the application program from being used with other vendor's devices which may not have any knowledge of the UnifiedPOS Service's need for this event.

See Also "Events" on page 15, **directIO** Method.

2.6.3 ErrorEvent

Updated in Release 1.13

<<event>> upos::events::ErrorEvent
ErrorCode: *int32* { read-only }
ErrorCodeExtended: *int32* { read-only }
ErrorLocus: *int32* { read-only }
ErrorResponse: *int32* { read-write }

Description Notifies the application that an error has been detected and a suitable response is necessary to process the error condition.

Attributes This event contains the following attributes:

Attribute	Type	Description
ErrorCode	*int32*	Error Code causing the error event. See the list of *ErrorCode*s under "Errors" on page 16.
ErrorCodeExtended	*int32*	Extended Error Code causing the error event. These values are device category specific.
ErrorLocus	*int32*	Location of the error. See values below.
ErrorResponse	*int32*	Error response, whose default value may be overridden by the application (i.e., this attribute is settable). See values below.

The *ErrorLocus* attribute has one of the following values:

Value	Meaning
EL_OUTPUT	Error occurred while processing asynchronous output.
EL_INPUT	Error occurred while gathering or processing event-driven input. No previously buffered input data is available.
EL_INPUT_DATA	Error occurred while gathering or processing event-driven input, and some previously buffered data is available.

The application's error event handler can set the *ErrorResponse* attribute to one of the following values: *(Updated in 1.13)*

Value	Meaning
ER_RETRY	Retry sending the data. The error state is exited. May be valid for some input devices when the locus is EL_INPUT, in which case the input is retried and the error state is exited. Typically valid for asynchronous output devices when the locus is EL_OUTPUT, in which case the asynchronous output is retried and the error state is exited. This is the default response when the locus is EL_OUTPUT.
ER_CLEAR	Valid for all loci: EL_INPUT, EL_INPUT_DATA, and EL_OUTPUT. Clear all buffered input or output data (including all asynchronous output). The error state is exited. This is the default response when the locus is EL_INPUT.
ER_CONTINUEINPUT	Only valid when the locus is EL_INPUT_DATA. Acknowledges that a data error has occurred and directs the Device to continue input processing. The Device remains in the error state and will deliver additional **DataEvent**s as directed by the **DataEventEnabled** property. When all input has been delivered and **DataEventEnabled** is again set to true, then another **ErrorEvent** is delivered with locus EL_INPUT. This is the default response when the locus is EL_INPUT_DATA.

Remarks This event is enqueued when an error is detected and the Device's **State** transitions into the error state. Input error events are not delivered until **DataEventEnabled** is true, so that proper application sequencing occurs.

Unlike a **DataEvent**, the Device does not disable further **DataEvent**s or input **ErrorEvent**s; it leaves the **DataEventEnabled** property value at true. Note that the application may set **DataEventEnabled** to false within its event handler if subsequent input events need to be disabled for a period of time.

See Also "Device Input Model" on page 18, "Error Handling" on page 19, "Device Output Models" on page 21.

2.6.4 OutputCompleteEvent

Updated in Release 1.13

<<event>> **upos::events::OutputCompleteEvent**
OutputID: ***int32*** **{ read-only }**

Description Notifies the application that the queued output request associated with the *OutputID* attribute has completed successfully.

Attribute This event contains the following attribute:

Attribute	Type	Description
OutputID	*int32*	The ID number of the asynchronous output request that is complete.

Remarks This event is enqueued after the requested data has been both sent and the UnifiedPOS Service has confirmation that is was processed by the device successfully.

See Also "Device Output Models" on page 21, **OutputID** Property.

2.6.5 StatusUpdateEvent

Updated in Release 1.9

<<event>> upos::events::StatusUpdateEvent
Status: *int32* { read-only }

Description Notifies the application when a device has detected an operation status change.

Attribute This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	Device category-specific status, describing the type of status change.

Release 1.3 and later – Power State Reporting

Power State Reporting, added in ***Release 1.3***, adds additional *Status* values of:

Value	Meaning
SUE_POWER_ONLINE	The device is powered on and ready for use. Can be returned if **CapPowerReporting** = PR_STANDARD or PR_ADVANCED.
SUE_POWER_OFF	The device is off or detached from the terminal. Can only be returned if **CapPowerReporting** = PR_ADVANCED.
SUE_POWER_OFFLINE	The device is powered on but is either not ready or not able to respond to requests. Can only be returned if **CapPowerReporting** = PR_ADVANCED.
SUE_POWER_OFF_OFFLINE	The device is either off or off-line. Can only be returned if **CapPowerReporting** = PR_STANDARD.

The common property **PowerState** is also maintained at the current power state of the device.

Release 1.9 and later – Update Firmware Reporting

The Update Firmware capability, added in ***Release 1.9***, adds the following *Status* values for communicating the status/progress of an asynchronous update firmware process:

Value	Meaning
SUE_UF_PROGRESS + 1 to 100	The update firmware process has successfully completed 1 to 100 percent of the total operation.
SUE_UF_COMPLETE	The update firmware process has completed successfully. The value of this constant is identical to SUE_UF_PROGRESS + 100.
SUE_UF_COMPLETE_DEV_NOT_RESTORED	The update firmware process succeeded, however the Service and/or the physical device cannot be returned to the state they were in before the update firmware process started. The Service has restored all properties to their default initialization values. To ensure consistent Service and physical device states, the application needs to **close** the Service, then **open**, **claim**, and enable again, and also restore all custom application settings.
SUE_UF_FAILED_DEV_OK	The update firmware process failed but the device is still operational.
SUE_UF_FAILED_DEV_UNRECOVERABLE	The update firmware process failed and the device is neither usable nor recoverable through software. The device requires service to be returned to an operational state.
SUE_UF_FAILED_DEV_NEEDS_FIRMWARE	The update firmware process failed and the device will not be operational until another attempt to update the firmware is successful.
SUE_UF_FAILED_DEV_UNKNOWN	The update firmware process failed and the device is in an indeterminate state.

Remarks This event is enqueued when a Device needs to alert the application of a device status change. Examples are a change in the cash drawer position (open vs. closed) or a change in a POS printer sensor (form present vs. absent).

When a device is enabled, the Control may deliver this event to inform the application of the device state. This behavior, however, is not required.

See Also "Events" on page 15, "Device Power Reporting Model" on page 22, **CapPowerReporting** Property, **CapUpdateFirmware** Property, **PowerNotify** Property.

34 Scale

34.1 General

This Chapter defines the Scale device category.

34.2 Summary

Properties (UML attributes)

Common	*Type*	*Mutability*	*Version*	*May Use After*
AutoDisable:	*boolean*	{ read-write }	1.3	open
CapCompareFirmwareVersion:	*boolean*	{ read-only }	1.9	open
CapPowerReporting:	*int32*	{ read-only }	1.3	open
CapStatisticsReporting:	*boolean*	{ read-only }	1.8	open
CapUpdateFirmware:	*boolean*	{ read-only }	1.9	open
CapUpdateStatistics:	*boolean*	{ read-only }	1.8	open
CheckHealthText:	*string*	{ read-only }	1.0	open
Claimed:	*boolean*	{ read-only }	1.0	open
DataCount:	*int32*	{ read-only }	1.3	open
DataEventEnabled:	*boolean*	{ read-write }	1.3	open
DeviceEnabled:	*boolean*	{ read-write }	1.0	open & claim
FreezeEvents:	*boolean*	{ read-write }	1.0	open
OutputID:	*int32*	{ read-only }	1.0	Not Supported
PowerNotify:	*int32*	{ read-write }	1.3	open
PowerState:	*int32*	{ read-only }	1.3	open
State:	*int32*	{ read-only }	1.0	--
DeviceControlDescription:	*string*	{ read-only }	1.0	--
DeviceControlVersion:	*int32*	{ read-only }	1.0	--
DeviceServiceDescription:	*string*	{ read-only }	1.0	open
DeviceServiceVersion:	*int32*	{ read-only }	1.0	open
PhysicalDeviceDescription:	*string*	{ read-only }	1.0	open
PhysicalDeviceName:	*string*	{ read-only }	1.0	open

Properties (Continued)

Specific	*Type*	*Mutability*	*Version*	*May Use After*
CapDisplay:	*boolean*	{ read-only }	1.2	open
CapDisplayText:	*boolean*	{ read-only }	1.3	open
CapFreezeValue	*boolean*	{ read-only }	**1.14**	open
CapPriceCalculating:	*boolean*	{ read-only }	1.3	open
CapReadLiveWeightWithTare	*boolean*	{ read-only }	**1.14**	open
CapSetPriceCalculationMode	*boolean*	{ read-only }	**1.14**	open
CapSetUnitPriceWithWeightUnit	*boolean*	{ read-only }	**1.14**	open
CapSpecialTare	*boolean*	{ read-only }	**1.14**	open
CapStatusUpdate:	*boolean*	{ read-only }	1.9	open
CapTarePriority	*boolean*	{ read-only }	**1.14**	open
CapTareWeight:	*boolean*	{ read-only }	1.3	open
CapZeroScale:	*boolean*	{ read-only }	1.3	open
AsyncMode:	*boolean*	{ read-write }	1.3	open
MaxDisplayTextChars:	*int32*	{ read-only }	1.3	open
MaximumWeight:	*int32*	{ read-only }	1.0	open
MinimumWeight:	*int32*	{ read-only }	**1.14**	open
SalesPrice:	*currency*	{ read-only }	1.3	open, claim, & enable
ScaleLiveWeight:	*int32*	{ read-only }	1.9	open
StatusNotify:	*int32*	{ read-write }	1.9	open
TareWeight:	*int32*	{ read-write }	1.3	open, claim, & enable
UnitPrice:	*currency*	{ read-write }	1.3	open, claim, & enable
WeightUnit:	*int32*	{ read-only }	1.0	open
ZeroValid	*boolean*	{ read-write }	1.13	open

Methods (UML operations)

Common

Name	*Version*
open (logicalDeviceName: *string* **):** **void { raises-exception }**	1.3
close ( ): **void { raises-exception, use after open }**	1.3
claim (timeout: *int32* **):** **void { raises-exception, use after open }**	1.3
release ( ): **void { raises-exception, use after open, claim }**	1.3
checkHealth (level: *int32* **):** **void { raises-exception, use after open, claim, enable }**	1.3

clearInput ( ): 1.3
void { raises-exception, use after open, claim }

clearInputProperties ( ): ***Not supported***
void { }

clearOutput ( ): ***Not supported***
void { }

directIO (command: *int32*, **inout data:** *int32*, **inout obj:** *object* **):** 1.3
void { raises-exception, use after open }

compareFirmwareVersion (firmwareFileName: *string*, **out result:** *int32* **):** 1.9
void { raises-exception, use after open, claim, enable }

resetStatistics (statisticsBuffer: *string* **):** 1.8
void { raises-exception, use after open, claim, enable }

retrieveStatistics (inout statisticsBuffer: *string* **):** 1.8
void { raises-exception, use after open, claim, enable }

updateFirmware (firmwareFileName: *string* **):** 1.9
void { raises-exception, use after open, claim, enable }

updateStatistics (statisticsBuffer: *string* **):** 1.8
void { raises-exception, use after open, claim, enable }

Specific

Name

displayText (data: *string* **):** 1.3
void { raises-exception, use after open, claim, enable }

doPriceCalculating (out weightValue: *int32,* **out tare:** *int32,* **1.14**
out unitPrice: *currency,* **out unitPriceX:** *currency,*
out weightUnitX: *int32,* **out weightNumeratorX:** *int32,*
out weightDenominatorX: *int32,* **out price:** *currency,*
timeout: *int32* **):**
void { raises-exception, use after open, claim, enable }

freezeValue (item: *int32,* **freeze:** *boolean* **):** **1.14**
void { raises-exception, use after open, claim, enable }

readLiveWeightWithTare (out weightData: *int32*, **out tare:** *int32,* **1.14**
timeout: *int32* **):**
void { raises-exception, use after open, claim, enable }

readWeight (inout weightData: *int32*, **timeout:** *int32* **):** 1.3
void { raises-exception, use after open, claim, enable }

setPriceCalculationMode (mode: *int32* **):** **1.14**
void { raises-exception, use after open, claim, enable }

setSpecialTare (mode: *int32,* **data:** *int32* **):** **1.14**
void { raises-exception, use after open, claim, enable }

setTarePrioity (priority: *int32* **):** **1.14**
void { raises-exception, use after open, claim, enable }

setUnitPriceWithWeightUnit (unitPrice: *currency,* **weightUnit:** *int32,* **1.14**
weightNumerator: *int32,* **weightDenominator:** *int32* **):**
void { raises-exception, use after open, claim, enable }

zeroScale ( ): 1.3
void { raises-exception, use after open, claim, enable }

Events (UML interfaces)

Name	*Type*	*Mutability*	*Version*
upos::events::DataEvent			1.3
Status:	*int32*	{ read-only }	
upos::events::DirectIOEvent			1.0
EventNumber:	*int32*	{ read-only }	
Data:	*int32*	{ read-write }	
Obj:	*object*	{ read-write }	
upos::events::ErrorEvent			1.3
ErrorCode:	*int32*	{ read-only }	
ErrorCodeExtended:	*int32*	{ read-only }	
ErrorLocus:	*int32*	{ read-only }	
ErrorResponse:	*int32*	{ read-write }	
upos::events::OutputCompleteEvent		***Not Supported***	
upos::events::StatusUpdateEvent			1.3
Status:	*int32*	{ read-only }	

34.3 General Information

The Scale programmatic name is “Scale.”

34.3.1 Capabilities

The scale Device has the following capability:

- Provides item weight to the application. The measure of weight may be in grams, kilograms, ounces, or pounds, depending upon the scale device.

The scale may have the following additional capabilities:

- Includes an integrated display with the current weight, or with the current weight plus application-specified text.
- Performs price calculations (weight X unit price) and returns the sale price. (This feature is mostly used in Europe at this time.)
- Supports application setting of tare weight.
- Supports application zeroing of the scale.

The following functionality is added for **Release 1.9**:

A scale device is used to obtain weight for two distinct purposes, legal weight for calculating price, and live weight for updating customer displays. Prior to Release 1.9, a good interface is provided for an application to obtain a legal weight, but no interface for obtaining a live weight existed. The following added functionality in **Release 1.9** formalizes an interface for obtaining scale status and live weight:

- A scale weight status update capability property, **CapStatusUpdate**.
- A scale weight status notify property, **StatusNotify**, to enable or disable weight status event notification.
- A **ScaleLiveWeight** property containing a value to be used for updating a customer display with the current scale weight.
- Extensions to the **readWeight** method and **StatusUpdateEvent** for scale weight status.

The following functionality is added for **Release 1.13**.

A writable property that controls the delivery of “0” as a valid weight for applications that need to report this as a legitimate value for the weight was added. When the property **ZeroValid** is true, the service is allowed to report “0” back to the application as a valid weight; when false, allows the service to be backward compatible by not allowing a “0” weight to be valid.

Changes in Release 1.14

The more sophisticated scales have the functionality to not only weigh items but to calculate the prices of the items in the scale and return to the application. Building on simple price calculation added in Release 1.3, Release 1.14 adds more complex price calculation allowing for multiple tare values and adding additional items on the scale which may have different pricing requirements. The new Properties and Methods are:

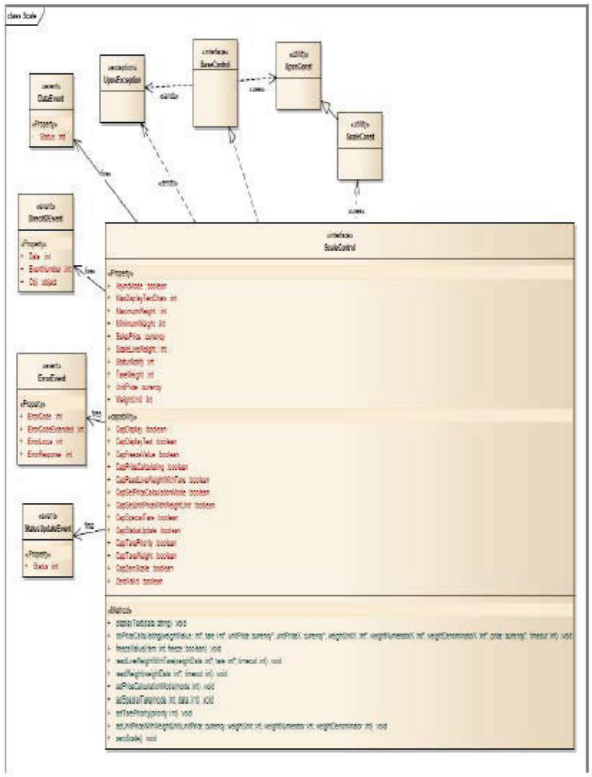
- A **CapFreezeValue** property to indicate if the scale supports the **freezeValue** method.
- A **CapReadLiveWeightWithTare** property to indicate if the scale supports live weight measurements incorporating a tare value.
- A **CapSetPriceCalculationMode** property to indicate if the scale supports different methods to calculate price.
- A **CapSetUnitPriceWithWeightUnit** property to indicate if the scale supports the ability to use different weight unit types apart from the default scale weight unit types; useful for more complext price calculations.
- A **CapSpecialTare** property to indicate if the scale supports the ability to use different tare values in replacement of or in addition to the default scale tare value used in determining the net weight.
- A **CapTarePriority** property to indicate if the scale supports the ability to use multiple tare values in a certain ranking order for the calculation of net weight and item price.
- A **MinimumWeight** property which contains the minimum value that the scale will use before it will register a valid weight read.
- A **doPriceCalculating** method that comprises the functionality of the readWeight method plus the ability to do price calculating. All the properties necessary to facilitate the price calculation are included in one method call.
- A **freezeValue** method to control the state of the tare and unit price values that the scale uses.
- A **readLiveWeightWithTare** method that allows the scale to return the live weight and the tare value; it may be used to display the tare value and weight value. In this method, the live weight is the stable net weight.

- A **setPriceCalculationMode** method to allow for different uses of the scale such as self service or operator attended modes.
- A **setSpecialTare** method that provides for different ways the scale can use the tare values in deteriming net weight and item price.
- A **setTarePriority** method that provides for ranking the order of tare values the scale can use in determining the net weight and item price.
- A **setUnitPriceWithWeightUnit** method that allows the scale to calculate the price of the item using other than the default scale parameter values.

34.4 Scale Class Diagram

Updated in Release 1.14

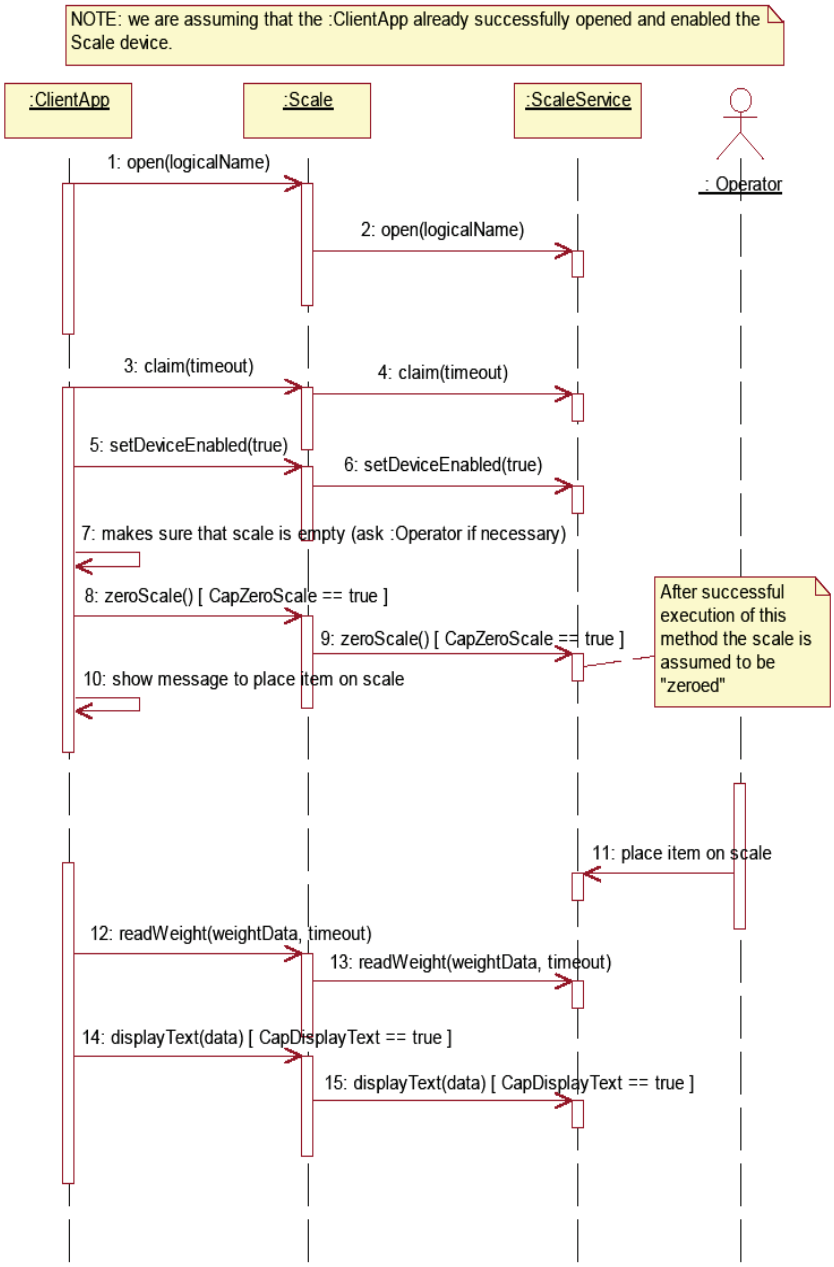
The following diagram shows the relationships between the Scale classes.



34.5 Scale Sequence Diagram

Added in Release 1.7

The following sequence diagram shows the typical synchronous usage of a Scale device.



34.5.1 Model

The general model of a scale is:

- A scale returns the weight of an item placed on its weighing surface.
- The primary scale method is **readWeight**. By default, it is performed synchronously. It returns after reading data from the scale; the weight is returned in the **readWeight**'s *weightData* parameter. If an error occurs or if the timeout elapses, a UposException will be thrown.
- *UnifiedPOS Release 1.3 and later - Asynchronous Input*

  If the **AsyncMode** property is true when **readWeight** is called, then the method is performed asynchronously. It initiates event driven input and returns immediately. The timeout parameter specifies the maximum time the application wants to wait for a settled weight. Additional points are:

  - If an error occurs while initiating event driven input (such as the device is offline), then a UposException is thrown. Otherwise, **readWeight** returns immediately to the application, and scale processing continues asynchronously.
  - If a settled weight is received, then a **DataEvent** is enqueued containing the weight data in the *Status* property.
  - If a scale error occurs (including a timeout with no settled weight), then an **ErrorEvent** is enqueued. The application event handler may retry the weighing process by setting the event's *ErrorResponse* property to ER_RETRY.
  - Only one asynchronous call to **readWeight** can be in progress at a time. An attempt to nest asynchronous scale operations will result in a UposException being thrown.
  - An asynchronous scale operation may be cancelled with the **clearInput** method.

For price-calculating scales, the application should set the **UnitPrice** property before calling **readWeight**. After a weight is read (and just before the **DataEvent** is delivered to the application, for asynchronous mode), the **SalesPrice** property is set to the calculated price of the item.

34.5.2 Device Sharing

The scale is an exclusive-use device, as follows:

- After opening the device, properties are readable.
- The application must claim the device before enabling it.
- The application must claim and enable the device before calling methods that manipulate the device.
- See the "Summary" table for precise usage prerequisites.

34.6 Properties (UML attributes)

34.6.1 AsyncMode Property

Added in Release 1.3

Syntax **AsyncMode:** ***boolean*** **{ read-write, access after open }**

Remarks If true, then the **readWeight** method will be performed asynchronously. If false, the **readWeight** method will be performed synchronously.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

34.6.2 CapDisplay Property

Syntax **CapDisplay:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale includes an integrated display that shows the current weight. If false, the application may need to show the current weight on another display.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **CapDisplayText** Property, **MaxDisplayTextChars** Property.

34.6.3 CapDisplayText Property

Added in Release 1.3

Syntax **CapDisplayText:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale includes an integrated display that shows the current weight and can also show text that describes the item being weighed. If false, extra text cannot be shown on the display.

If true, then **CapDisplay** must also be true.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **CapDisplay** Property, **MaxDisplayTextChars** Property.

34.6.4 CapFreezeValue Property

Added in Release 1.14

Syntax **CapFeeezeValue:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale supports the ability to determine and control the state and values of the tare and unit price that it uses after a **readWeight** or **doPriceCalculating** method call. If false, the scale does not support the **freezeValue** method.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **freezeValue** Method, **readWeight** Method, **doPriceCalculating** Method.

34.6.5 CapPriceCalcuating Property

Added in Release 1.3

Syntax **CapPriceCalculating:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale can calculate prices. If false, the scale only returns a weight.

For price calculating scales the calculation unit is in the scale rather than in the data-receiving terminal.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **readWeight** Method, **WeightUnit** Property, **UnitPrice** Property, **SalesPrice** Property.

34.6.6 CapReadLiveWeightWithTare Property

Added in Release 1.14

Syntax **CapReadLiveWeightWithTare:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale supports the ability to return the *weightData* and the *tare* value with the **readLiveWeightWithTare** method.

If false, the scale does not support the **readLiveWeightWith Tare** method.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **readLiveWeightWithTare** Method.

34.6.7 CapSetPriceCalculationMode Property

Added in Release 1.14

Syntax **CapSetPriceCalculationMode:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale can utilize different methods for calculating the price of a weighed item on the scale. This may be useful, for example, to determine the pricing information for a produce label. If false, the scale does not support the **setPriceCalculationMode** method.

For price calculating scales this functionality is resident in the scale rather than in the data-receiving terminal.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **setPriceCalculationMode** Method, **doPriceCalculating** Method, **WeightUnit** Property, **UnitPrice** Property, **SalesPrice** Property.

34.6.8 CapSetUnitPriceWithWeightUnit Property

Added in Release 1.14

Syntax **CapSetUnitPriceWithWeightUnit:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale can support a method to associate a unit price with a specific weight unit measure that is different from the default weight measure unit for the scale. If false, the scale can only associate a unit price with a preset weight measure unit.

For price calculating scales this functionality is resident in the scale rather than in the data-receiving terminal.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **setUnitPriceWithWeightUnit** Method, **readWeight** Method, **doPriceCalculating** Method, **WeightUnit** Property, **UnitPrice** Property, **SalesPrice** Property.

34.6.9 CapSpecialTare Property

Added in Release 1.14

Syntax **CapSpecialTare:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the scale supports special tare weight components that can be used in the calculations to determine the scale net weight. If false, the scale may only support standard scale tare net weight calculations.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **setSpecialTare** Method.

34.6.10 CapStatusUpdate Property

Added in Release 1.9

Syntax CapStatusUpdate: *boolean* { read-only, access after open }

Remarks If true, then the scale is capable of providing scale weight status with **StatusUpdateEvent**s. This property is initialized by the **open** method.

If true when the device is enabled, an immediate **StatusUpdateEvent** will be generated to tell the application the current state of the scale.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **ScaleLiveWeight** Property, **StatusNotify** Property.

34.6.11 CapTarePriority Property

Added in Release 1.14

Syntax **CapTarePriority: *boolean* { read-only, access after open }**

Remarks If true, the scale supports the ability to set the order in which multiple tare weight components can be applied in the calculations used to determine the scale net weight. If false, the scale does not support this **setTarePriority** method.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **setTarePriority** Method.

34.6.12 CapTareWeight Property

Added in Release 1.3

Syntax **CapTareWeight: *boolean* { read-only, access after open }**

Remarks If true, the scale includes setting a tare value. If false, the scale does not support tare values.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also TareWeight Property.

34.6.13 CapZeroScale Property

Added in Release 1.3

Syntax **CapZeroScale:** ***boolean*** **{ read-only, access after open }**

Remarks If true, the application can set the scale weight to zero. If false, the scale does not support programmatic zeroing.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **zeroScale** Method.

34.6.14 MaxDisplayTextChars Property

Added in Release 1.3

Syntax **MaxDisplayTextChars:** ***int32*** **{ read-only, access after open }**

Remarks Holds the number of characters that may be displayed on an integrated display for the text which describes an article.

If **CapDisplayText** is false, then the device does not support text displaying and this property is always zero.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **CapDisplay** Property, **CapDisplayText** Property.

34.6.15 MaximumWeight Property

Syntax **MaximumWeight:** ***int32*** **{ read-only, access after open }**

Remarks Holds the maximum weight measurement possible from the scale. The measurement unit is available via the **WeightUnit** property.

This property has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005.

The value held by this property must be processed considering the value returned by the **WeightUnit** property.

This property is initialized by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **WeightUnit** Property.

34.6.16 MinimumWeight Property

Added in Release 1.14

Syntax **MinimumWeight:** ***int32*** **{ read-only, access after open }**

Remarks Holds the minimum weight measurement possible from the scale. The measurement unit is available via the **WeightUnit** property.

This property has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 5 represents 0.005.

The value held by this property must be processed considering the value returned by the **WeightUnit** property.

This property is initialized by the **open** method. The minimum weight depends upon the operation mode of the scale (see **setPriceCalculationMode**).

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **WeightUnit** Property.

34.6.17 SalesPrice Property

Updated in Release 1.6

Syntax **SalesPrice:** ***currency*** **{ read-only, access after open }**

Remarks Holds the sales price read from the scale for price calculating scales. For price calculating scales the scale calculates this value during the process of weighing by multiplying the **UnitPrice** property by the acquired weight. This property is a monetary value stored using an implied four decimal places. For example, an actual value of 12345 represents 1.2345.

This property is set before the **readWeight** or **doPriceCalculating** methods return (in synchronous mode) or the **DataEvent** is delivered (in asynchronous mode).

If **CapPriceCalculating** is false, then the device is not a price calculating scale and **SalesPrice** is always zero.

This property is initialized to zero when the device is first enabled following the **open** method. (In releases prior to 1.5, this description stated that initialization took place by the **open** method. In Release 1.5, it was updated for consistency with other devices.)

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **readWeight** Method, **doPriceCalculating** Method, **setUnitPriceWithWeightUnit** Method, **WeightUnit** Property, **CapPriceCalculating** Property, **UnitPrice** Property.

34.6.18 ScaleLiveWeight Property

Updated in Release 1.14

Syntax ScaleLiveWeight: *int32* { read-only, access after open-claim-enable }

Remarks Contains the returned value for the weight measured by the scale if the **StatusUpdateEvent** *Status* is set to SCAL_SUE_STABLE_WEIGHT, else zero.

The property is set before the **readLiveWeightWithTare** method returns when **AsyncMode** = false or before the **DataEvent** is delivered when **AsyncMode** = true.

The weight has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005.

It is suggested that an application use the weight in this property only for display purposes. For a weight to use for sale purposes, it is suggested that the application call the **readWeight** or the **doPriceCalculating** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **"Device Input Model"** on page 18, **CapStatusUpdate** Property, **StatusNotify** Property, **readLiveWeightWithTare** method.

34.6.19 StatusNotify Property

Updated in Release 1.10

Syntax StatusNotify: *int32* { read-write, access after open }

Remarks Scale weight state notification can only be set by the application if the capability **CapStatusUpdate** is true. The **StatusNotify** values are:

Value	Meaning
SCAL_SN_DISABLED	The Control will not provide any scale weight state notifications to the application or set any related *ErrorCodeExtended* values. No scale weight state notification **StatusUpdateEvent**s will be fired, and **ScaleLiveWeight** may not be set.
SCAL_SN_ENABLED	The Control will fire scale weight state notification **StatusUpdateEvent**s and update the **ScaleLiveWeight** property beginning when **DeviceEnabled** is set true. The level of functionality depends upon **CapStatusUpdate**.

StatusNotify may only be set while the device is disabled, that is, while **DeviceEnabled** is false. This property is initialized to SCAL_SN_DISABLED by the **open** method. This value provides compatibility with earlier releases.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	One of the following errors occurred: • The device is already enabled. • **CapStatusUpdate** is false.

See Also **CapStatusUpdate** Property, **ScaleLiveWeight** Property.

34.6.20 TareWeight Property

Updated in Release 1.14

Syntax **TareWeight:** ***int32*** **{ read-write, access after open-claim-enable }**

Remarks Holds the tare weight of scale data. This property has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005. The measured unit is specified in the **WeightUnit** property. If **CapTareWeight** is false, then the device does not support setting of a tare value and this property is always zero.

TareWeight is not included in the item weight returned by the **readWeight** method. It is updated by the **doPriceCalculating** method.

This property is initialized to the scale's default tare weight (usually zero), when the device is first enabled following the **open** method. (In releases prior to 1.5, this description stated that initialization took place by the **open** method. In Release 1.5, it was updated for consistency with other devices.)

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapTareWeight** is false or an invalid tare value was specified.

See Also **readWeight** Method, **doPriceCalculating** method, **WeightUnit** Property, **CapTareWeight** Property.

34.6.21 UnitPrice Property

Updated in Release 1.14

Syntax **UnitPrice:** ***currency*** **{ read-write, access after open-claim-enable }**

Remarks Holds the unit price of the article to be weighed. For price calculating scales this property is to be set before calling the **readWeight** method. It is updated by the **doPriceCalculating** method. During weighing, the scale sets the **SalesPrice** property to the product of the item's weight and this property. This property is a monetary value stored using an implied four decimal places. For example, an actual value of 12345 represents 1.2345.

If **CapPriceCalculating** is false, then setting of a unit price is not supported and this property is always zero.

This property is initialized to zero when the device is first enabled following the **open** method. (In releases prior to 1.5, this description stated that initialization took place by the **open** method. In Release 1.5, it was updated for consistency with other devices.)

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapPriceCalculating** is false or an invalid price was specified.

See Also **readWeight** Method, **doPriceCalculating** method, **WeightUnit** Property, **CapPriceCalculating** Property, **SalesPrice** Property.

34.6.22 WeightUnit Property

Syntax WeightUnit: *int32* { read-only, access after open }

Remarks Holds the unit of weight of scale data, and has one of the following values:

Value	Meaning
SCAL_WU_GRAM	Unit is a gram.
SCAL_WU_KILOGRAM	Unit is a kilogram (= 1000 grams).
SCAL_WU_OUNCE	Unit is an ounce.
SCAL_WU_POUND	Unit is a pound (= 16 ounces).

This property is initialized to the scale's weight unit by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

34.6.23 ZeroValid Property

Added in Release 1.13

Syntax ZeroValid: *boolean* { read-write, access after open }

Remarks If true, then the **readWeight** method will return zero (0.00) as a valid stable weight.

If false, then the **readWeight** method will not return zero as a valid stable weight.

This property is initialized to false by the **open** method.

Errors A UposException may be thrown when this property is accessed. For further information, see **"Errors"** on page 16.

See Also **readWeight** Method.

34.7 Methods (UML operations)

34.7.1 displayText Method

Updated in Release 1.7

Syntax **displayText (data: *string*):**
void { raises-exception, use after open-claim-enable }

Parameter	Description
data[1]	The string of characters to display.

1. In the **OPOS** environment, the format of this data depends upon the value of the **BinaryConversion** property. See **BinaryConversion** property in Annex A.

Remarks If **CapDisplayText** is true, updates the text shown on the integrated display. Calling this method with an empty string (“”) will clear the display.

Errors A UposException may be thrown when this method is invoked. For further information, see **“Errors”** on page 16.

Some possible values of the exception’s *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	An invalid text was specified -- the text contains more characters than **MaxDisplayTextChars**, or **CapDisplayText** is false.

See Also **CapDisplay** Property, **CapDisplayText** Property, **MaxDisplayTextChars** Property.

34.7.2 doPriceCalculating Method

Added in Release 1.14

Syntax

doPriceCalculating (out weightData: *int32*, out tare: *int32,*
out unitPrice: *currency,* out unitPriceX: *currency,* out weightUnitX: *int32,*
out weightNumeratorX: *int32,* out weightDenominatorX: *int32,*
out price: *currency,* timeout: *int32*):
void { raises-exception, use after open-claim-enable }

Parameter	Description
weightData	The value for the net weight in the price calculation algorithm. If in asynchronous mode (**AsyncMode** is true), the returned value is zero.
tare	The value used to determine the item net weight in the price calculation algorithm. If in asynchronous mode (**AsyncMode** is true), the returned value is zero.
unitPrice	The cost per measurement unit that is used in the price calcuation algorithm. The measurement unit is the same as that in the scale **WeightUnit** property. If in asynchronous mode (**AsyncMode** is true), the returned value is zero.
unitPriceX	The cost per measurement unit that is used in the price calcuation algorithm that comes from the **setUnitPriceWithWeightUnit** method. The measurement unit is the same as that in the **setUnitPriceWithWeightUnit** method parameter, *weightUnit*.
weightUnitX	The value representing the unit of weight that differs from the default value for the scale and is the same as **setUnitPriceWithWeightUnit** method parameter, *weightUnit*.

weightNumeratorX	The dividend which is the weight value based on the **setUnitPriceWithWeightUnit** method parameter, *weightNumerator*.
weightDenominatorX	The divisor which is the weight value based on the **setUnitPriceWithWeightUnit** method parameter, *weightDenominator*.
price	The calculated monetary value for the item on the scale in the price calculation algorithm. If in asynchronous mode (**AsyncMode** is true), the returned value is zero.
timeout	In synchronous mode the number of milliseconds to wait for a settled weight before failing the method. If in asynchronous mode (**AsyncMode** is true), the *timeout* value is ignored.

Remarks

This method is used to have the scale calculate and return the price of the item(s) on it allowing for multiple price determing factors.

In synchronous mode (**AsyncMode = false**), this method starts the read weight process and when a stable weight is obtained, does a price calculation. Upon successful completion, the **ScaleLiveWeight**, **TareWeight**, **UnitPrice**, and **SalesPrice** properties are updated; the values for *weightData, tare, unitPrice, unitPriceX, weightUnitX, weightNumeratorX, weightDenominatorX,* and the resultant *price* are returned.

In asynchronous mode (**AsyncMode = true**), the weighing and subsequent price calcualtion is done asynchronously. The method returns immediately with the return values for *weightData, tare, unitPrice, unitPriceX, weightUnitX, weightNumeratorX, weightDenominatorX* and resultant *price* set as noted in table above.

Upon completion of the price calculating process, the **ScaleLiveWeight**, **TareWeight**, **UnitPrice**, and **SalesPrice** properties are updated and a **DataEvent** is delivered.

The weight returned, *weightData* and **ScaleLiveWeight,** has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005.

Errors

A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	An invalid *timeout* parameter was specified.
E_BUSY	An asynchronous **doPriceCalculating** method is in progress.
E_TIMEOUT	If **ZeroValid** is false, a stable non-zero weight was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false). If **ZeroValid** is true, a stable weight (including a zero weight) was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false).
E_EXTENDED	*ErrorCodeExtended* = ESCAL_OVERWEIGHT: The weight was over **MaximumWeight.** This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDERWEIGHT: The weight was under the **MinimumWeight.** This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDER_ZERO: The scale is reporting a weight that is less than zero due to a calibration error. The scale should be recalibrated. This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_SAME_WEIGHT: The scale is reporting that the item/weight on the scale is identical to the previously reported item/weight; i.e., the item has not been removed from the scale. This can only be returned if **AsyncMode** is false and the scale hardware directly supports this capability.

See Also **setUnitPriceWithWeightUnit** method, **UnitPrice** Property, **WeightUnit** Property, **CapPriceCalculating** Property, **CapSetPriceCalculationMode** property, **SalesPrice** Property, **TareWeight** Property, **ZeroValid** Property.

34.7.3 freezeValue Method

Added in Release 1.14

Syntax

freezeValue (item: ***int32*****, freeze:** ***boolean*** **):**
void { raises-exception, use after open-claim-enable }

Parameter	Description
item	The bitwise value setting the state of the selected parameter item(s).
freeze	If the *freeze* value is true, the representative *item* is not automatically set to zero; as an example after a **readWeight** method call and the weight is removed. Note: In this example, to delete the specific item without calling the **readWeight** method, make a **freezeValue** method call with the *freeze* value set to false. If the *freeze* value is set to false, the *item* is automatically set to zero after a **readWeight** method call and the removal of the weight.

Value	Description
SCAL_SFR_MANUAL_TARE	Freezes a manual tare
SCAL_SFR_WEIGHTED_TARE	Freezes a weighted tare
SCAL_SFR_PERCENT_TARE	Freezes a percentage tare
SCAL_SFR_UNITPRICE	Freezes the unit price

Remarks The **freezeValue** method performs a bitwise logical OR function to determine the state of the *item*(s) selected after a **readWeight** or a **doPriceCalculating** method call is processed. If the representative *item* bit value is set to true, then the scale will not clear (set to zero) the associated tare values and/or unit price.
If the representative *item* bit value is set to false, then the scale will clear (set to zero) the associated tare values and/or unit price.

Errors A UposException may be thrown when this method is invoked. For further information, see **" Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The current state of the scale device does not allow the freezing of the requested tare or unit price value.

See Also doPriceCalculating method, readWeightWithTare method, setSpecialTare method

34.7.4 readLiveWeightWithTare Method

Added in Release 1.14

Syntax

```
readLiveWeightWithTare ( out weightData: int32, out tare: int32,
timeout: int32 ):
        void { raises-exception, use after open-claim-enable }
```

Parameter	Description
weightData	If in synchronous mode (**AsyncMode** is false), contains the returned value for the net weight calculated by the scale. If in asynchronous mode (**AsyncMode** is true) the returned value is zero.
tare	The value used to calculate the net weight. If in asynchronous mode (**AsyncMode** is true), the returned value is zero.
timeout	In synchronous mode the number of milliseconds to wait for a settled weight before failing the method. If in asynchronous mode (**AsyncMode** is true), the *timeout* value is ignored.

Remarks This method is used to determine the value for the displaying the net weight.
In synchronous mode (**AsyncMode** is false), this method starts the read weight process and when a stable weight is obtained, does a net weight calculation. Upon successful completion, the **ScaleLiveWeight** and **TareWeight** properties are updated and the values for *weightData and tare* are returned.

In asynchronous mode (**AsyncMode** is true), the weighing and subsequent net weight calculation is done asynchronously. The method returns immediately with the return values for *weightData and tare* set as noted above. Upon completion of this method, the **ScaleLiveWeight** and **TareWeight** properties are updated and a **DataEvent** is delivered.

The weight returned, *weightData* and **ScaleLiveWeight,** has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005.

Errors A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	An invalid *timeout* parameter was specified.
E_BUSY	An asynchronous **readWeightWithTare** method is in progress.
E_TIMEOUT	If **ZeroValid** is false, a stable non-zero weight was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false). If **ZeroValid** is true, a stable weight (including a zero weight) was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false).
E_EXTENDED	*ErrorCodeExtended* = ESCAL_OVERWEIGHT: The weight was over **MaximumWeight.** This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDERWEIGHT: The weight was under the **MinimumWeight.** This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDER_ZERO: The scale is reporting a weight that is less than zero due to a calibration error. The scale should be recalibrated. This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_SAME_WEIGHT: The scale is reporting that the item/weight on the scale is identical to the previously reported item/weight; i.e., the item has not been removed from the scale. This can only be returned if **AsyncMode** is false and the scale hardware directly supports this capability.

See Also **ScaleWeight** Property, **TareWeight** Property, **ZeroValid** Property, **readWeight** method.

34.7.5 readWeight Method

Syntax

readWeight (inout weightData: *int32*, timeout: *int32*):
void { raises-exception, use after open-claim-enable }

Parameter	Description
weightData	If **AsyncMode** is false, contains the returned value for the weight measured by the scale, else zero.
timeout	The number of milliseconds to wait for a settled weight before failing the method. If zero, the method attempts to read the scale weight, then returns the appropriate status immediately. If FOREVER (-1), the method waits as long as needed until a weight is successfully read or an error occurs.

Remarks

Reads a weight from the scale.

The weight returned, *weightData*, has an assumed decimal place located after the "thousands" digit position. For example, an actual value of 12345 represents 12.345, and an actual value of 5 represents 0.005.

Release 1.2
The weighing process is performed synchronously and the method will return after finishing the weighing process. The weight is returned in the *weightData* parameter.

Release 1.3 and later
If **AsyncMode** is false, then **readWeight** operates synchronously, as with earlier releases.

Release 1.13 and later
If the **ZeroValid** property is true, the scale service will return zero as a valid weight. If this property is false, then the service will behave as prior to release 1.13, namely zero on the scale platter will result in E_TIMEOUT. This property is initialized to false by the **open** method.

If **AsyncMode** is true, the weighing process is performed asynchronously. The method will initiate a read, then return immediately. Once the weighing process is complete, a **DataEvent** is delivered with the item's weight contained in the event's *Status* property.

Errors

A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	An invalid *timeout* parameter was specified.
E_BUSY	An asynchronous **readWeight** is in progress.
E_TIMEOUT	If **ZeroValid** is false, a stable non-zero weight was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false). If **ZeroValid** is true, a stable weight (including a zero weight) was not available before *timeout* milliseconds elapsed (only if **AsyncMode** is false).
E_EXTENDED	*ErrorCodeExtended* = ESCAL_OVERWEIGHT: The weight was over **MaximumWeight.** This can only be returned if **AsyncMode** is false.

The following standard extended error codes have been **added in Release 1.14**

E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDERWEIGHT: The weight was under the **MinimumWeight.** This can only be returned if **AsyncMode** is false.

The following standard extended error codes have been **added in Release 1.9** as possible values of the exception's *ErrorCode* property:

E_EXTENDED	*ErrorCodeExtended* = ESCAL_UNDER_ZERO: The scale is reporting a weight that is less than zero due to a calibration error. The scale should be re-calibrated. This can only be returned if **AsyncMode** is false.
E_EXTENDED	*ErrorCodeExtended* = ESCAL_SAME_WEIGHT: The scale is reporting that the item/weight on the scale is identical to the previously reported item/weight; i.e., the item has not been removed from the scale. This can only be returned if **AsyncMode** is false and the scale hardware directly supports this capability.

See Also **UnitPrice** Property, **WeightUnit** Property, **CapPriceCalculating** Property, **SalesPrice** Property, **TareWeight** Property, **ZeroValid** Property.

34.7.6 setPriceCalculationMode Method

Added in Release 1.14

Syntax

setPriceCalculationMode (mode: *int32*):
void { raises-exception, use after open-claim-enable }

Parameter	Description
mode	The operation functionality selected for the scale.

Value	Description
SCAL_PCM_PRICE_LABELING	Set scale to price labeling mode.The scale has a printer and is capable of printing price labels.
SCAL_PCM_SELF_SERVICE	Set scale to self service mode. The customer is weighing the products placed on the scale.
SCAL_PCM_OPERATOR	Set scale to operator mode. The operator is using the scale and weighing the items for the customer.

Remarks This method allows for various modes of operation based upon the user and provides for the corresponding rules for price calculations.

Errors A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page Intro-21.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The current state of the scale device does not allow this type of functionality or invalid parameters were received.

See Also CapSetPriceCalculationMode Property

34.7.7 setSpecialTare Method

Added in Release 1.14

Syntax **setSpecialTare (mode: *int32*, data: *int32*):**
void { raises-exception, use after open-claim-enable }

Parameter	Description
mode	Select the tare mode that is to be modified.
data	Provides additional information specific to the mode selected to determine the characteristics for the tare that is to be modified.

Value	Description
SCAL_SST_DEFAULT	The data argument is interpreted as a weight value. For instance, a value of 12345 means 12.345 kg. The measured unit is specified in the **WeightUnit** property. The data argument will be used as the **TareWeight** for the price calculation.
SCAL_SST_MANUAL	The data argument is interpreted as a weight value. For instance, a value of 12345 means 12.345 kg. The measured unit is specified in the **WeightUnit** property. The data argument will be used as the **TareWeight** for the price calculation. A data value of zero disables the tare immediately and deletes the tare value.
SCAL_SST_PERCENT	The data argument is interpreted as a percent value. For instance a value of 99999 means 999.99%. A data value of zero disables the tare immediately and deletes the tare value.
SCAL_SST_WEIGHTED	If there is a weight on the scale the data argument is ignored and the weight from the scale will be used as the **TareWeight** for the next price calculation. When there is no weight on the scale the weighted tare is deleted.

Remarks The **TareWeight** used by the scale usually differs from the *data* parameter and depends upon the rounding rules of the scale. The exact value for *data* is returned by the **doPriceCalculation** method. If a tare is set, additional **setSpecialTare** calls with the same mode parameter are accepted and will update the new *data* value. Other values of the mode parameter may be accepted and depend upon the tare priority indicated by the **setTarePriority** or the influence of local jurisdictional laws.
In addition, the tare value might be deleted automatically if this action is required as a result of a prior **freezeValue** method call.

Errors A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The current state of the scale device does not allow this operation.

See Also CapSpecialTare Property, setTarePriority method, readWeight method, doPriceCalculation method, freezeValue method

34.7.8 setTarePriority Method

Added in Release 1.14

Syntax

setTarePriority(priority: *int32*):
void { raises-exception, use after open-claim-enable }

Parameter	Description
priority	The sequence in which a tare value is used when determining the net weight.

Value	Description
SCAL_STP_FIRST	If a tare is active, no other tare can be selected until the current tare is disabled.
SCAL_STP_NONE	Any tare can replace the currently selected tare.

Remarks

This method provides the mechanism to select the set of rules that can be used to control the prioritization of the tare component for net weight calculations.

Errors

A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The current state of the scale device does not allow this operation.

See Also

CapTarePriority Property

34.7.9 setUnitPriceWithWeightUnit Method

Added in Release 1.14

Syntax

setUnitPriceWithWeightUnit (unitPrice: *currency*, weightUnit: *int32,*
weightNumerator: *int32,* weightDenominator: *int32*):
void { raises-exception, use after open-claim-enable }

Parameter	Description
unitPrice	The cost per unit price as calculated by this method.
weightUnit	The value representing the new unit of weight that differs from the default value for the scale.
weightNumerator	The dividend which is the weight value based on the current unit weight.
weightDenominator	The divisor which is the weight value based on the new unit weight.

weightUnit Value	Description
SCAL_WU_GRAM	Units of weight specified in grams
SCAL_WU_KILOGRAM	Units of weight specified in kilograms
SCAL_WU_OUNCE	Units of weight specified in ounces
SCAL_WU_POUND	Units of weight specified in pounds

Remarks This method can be used to calculate a new *unitPrice* based upon a conversion factor that translates the old per *unitPrice* into a new per *unitPrice.*
For an example:
The tags at a chocolate shop are based upon 100 g instead of 1kg. The conversion calculation can be done by the scale instead of forcing the application to normalize every tag to kg. The scale works with kg by default. The application has provided the unit price of chocolate to be 2.55 Euros per 100 g. The correct weighing can be configured by:
setUnitPriceWithWeightUnit (2.55, **SCAL_WU_GRAM**, 100, 1);

Errors A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	The current state of the scale device does not allow this function of the scale or wrong parameters have been used.

See Also WeightUnit Property

34.7.10 zeroScale Method

Updated in Release 1.10

Syntax
zeroScale ( ):
void { raises-exception, use after open-claim-enable }

Remarks If **CapZeroScale** is true, sets the current scale weight to zero. It may be used for initial calibration, or to account for tare weight on the scale.

Errors A UposException may be thrown when this method is invoked. For further information, see **"Errors"** on page 16.

Some possible values of the exception's *ErrorCode* property are:

Value	Meaning
E_ILLEGAL	**CapZeroScale** is false.
E_BUSY	An asynchronous **readWeight** is in progress.

See Also **CapZeroScale** Property.

34.8 Events (UML interfaces)

34.8.1 DataEvent

Added in Release 1.3

<< event >> upos::events::DataEvent
Status: *int32* { read-only }

Description Notifies the application that an asynchronous **readWeight** has completed.

Attributes This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	The weight of the item.

Remarks If the scale is a price calculating scale, the unit price is placed in the **UnitPrice** property and the calculated sales price is placed in the **SalesPrice** property before this event is delivered.

See Also **"Events"** on page 15.

34.8.2 DirectIOEvent

<< event >> upos::events::DirectIOEvent
EventNumber: *int32* { read-only }
Data: *int32* { read-write }
Obj: *object* { read-write }

Description Provides Service information directly to the application. This event provides a means for a vendor-specific Scale Service to provide events to the application that are not otherwise supported by the Control.

Attributes This event contains the following attributes:

Attribute	Type	Description
EventNumber	*int32*	Event number whose specific values are assigned by the Service.
Data	*int32*	Additional numeric data. Specific values vary by the *EventNumber* and the Service. This property is settable.
Obj	*object*	Additional data whose usage varies by the *EventNumber* and Service. This property is settable.

Remarks This event is to be used only for those types of vendor specific functions that are not otherwise described. Use of this event may restrict the application program from being used with other vendor's Scale devices which may not have any knowledge of the Service's need for this event.

See Also **"Events"** on page 15, **directIO** Method.

34.8.3 ErrorEvent

Updated in Release 1.10

<< event >> upos::events::ErrorEvent
ErrorCode: ***int32*** **{ read-only }**
ErrorCodeExtended: ***int32*** **{ read-only }**
ErrorLocus: ***int32*** **{ read-only }**
ErrorResponse: ***int32*** **{ read-write }**

Description Notifies the application that a scale device error has been detected and a suitable response by the application is necessary to process the error condition.

Attributes This event contains the following attributes:

Attribute	Type	Description
ErrorCode	*int32*	Error code causing the error event. See list of *ErrorCode*s on page 0-21.
ErrorCodeExtended	*int32*	Extended error code causing the error event. It may contain a Service-specific value.
ErrorLocus	*int32*	Location of the error. See values below.
ErrorResponse	*int32*	Error response, whose default value may be overridden by the application (i.e., this property is settable). See values below.

The *ErrorLocus* property has one of the following values:

Value	Meaning
EL_INPUT	Error occurred while gathering or processing event-driven input. No previously buffered input data is available.
EL_INPUT_DATA	Error occurred while gathering or processing event-driven input, and some previously buffered data is available.

The contents of the *ErrorResponse* property are preset to a default value, based on the *ErrorLocus*. The application's error processing may change *ErrorResponse* to one of the following values:

Value	Meaning
ER_RETRY	Retry the asynchronous input. The error state is exited.
ER_CLEAR	Clear the buffered input data. The error state is exited. Default when locus is EL_INPUT.
ER_CONTINUEINPUT	Use only when locus is EL_INPUT_DATA. Acknowledges the error and directs the Device to continue processing. The Device remains in the error state, and will deliver additional **DataEvent**s as directed by the **DataEventEnabled** property. When all input has been delivered and **DataEventEnabled** is again set to true, then another **ErrorEvent** is delivered with locus EL_INPUT. Default when locus is EL_INPUT_DATA.

Remarks Enqueued when an error is detected while trying to read scale data. This event is not delivered until **DataEventEnabled** is true, so that proper application sequencing occurs.

See Also **"Events"** on page 15.

34.8.4 StatusUpdateEvent

Updated in Release 1.10

<< event >> upos::events::StatusUpdateEvent
Status: *int32* { read-only }

Description Notifies the application that there is a change in the power status of a Scale device.

If the **StatusNotify** property is SCAL_SN_ENABLED, this event can also notify the application that there is a change in the Scale device weight.

If the property **StatusNotify** is true when the scale is enabled, an immediate **StatusUpdateEvent** should be generated to notify the application of the current state of the scale.

Attributes This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	Reports a change in the power state of a Scale device. ***Note that Release 1.3*** added Power State Reporting with additional *Power reporting* **StatusUpdateEvent** *values.* The Update Firmware capability, added in ***Release 1.9***, added additional *Status* values for communicating the status/progress of an asynchronous update firmware process. See "**StatusUpdateEvent**" description in Chapter 2.

Added in Release 1.9 and delivered if **StatusNotify** is set to SCAL_SN_ENABLED.

Value	Meaning
SCAL_SUE_STABLE_WEIGHT	Scale weight is stable. The **ScaleLiveWeight** property is updated before event delivery.
SCAL_SUE_WEIGHT_UNSTABLE	Scale weight is unstable.
SCAL_SUE_WEIGHT_ZERO	Scale weight is zero.
SCAL_SUE_WEIGHT_OVERWEIGHT	Scale weight is overweight.
SCAL_SUE_WEIGHT_UNDERWEIGHT	Scale weight is underweight.
SCAL_SUE_NOT_READY	Scale is not ready to weigh.
SCAL_SUE_WEIGHT_UNDER_ZERO	Scale weight is under zero.

Remarks Enqueued when the Scale device detects a power state change or a status change.

See Also **"Events"** on page 15, **ScaleLiveWeight** Property, **StatusNotify** Property.